

2. Runde

des

43. Bundeswettbewerbs Informatik

Aufgabe 2: Simultane Labyrinth

Bearbeitet

von

Florian Bange
Teilnahme-ID: 74405

28. April 2025

Inhaltsverzeichnis

1	Einleitung	2
2	Mathematische Formulierung und Notation	3
3	Existenz einer Loesung und Approximationsverfahren	4
3.1	Existenz eines Loesungsweges	4
3.2	Approximationsverfahren - Verbindung zweier optimaler Wege	5
3.3	Analyse der Platz- und Zeitkomplexitaet der Algorithmen	6
4	Verfahren fuer optimale Loesungen	7
4.1	Loesungsverfahren fuer optimale Loesungswege	7
4.2	Optimierungen des Loesungsverfahrens	7
4.3	Analyse der Platz- und Zeitkomplexitaet der Loesungsverfahren	8
5	Erweiterung I - k Labyrinth	9
5.1	Verallgemeinerung des Approximationsverfahrens	9
5.2	Verallgemeinerung des Verfahrens fuer optimale Loesungswege	9
5.3	Beweis der NP-Schwere des Problems mit k Labyrinth	9
6	Implementierung	12
6.1	Darstellung der Labyrinth	12
6.2	Weitere Implementierungsdetails	12
7	Evaluierung anhand von Beispielen	13
7.1	Zusammenfassung der Ergebnisse	13
7.2	Die Beispiele der BwInf-Website	14
7.3	Fazit der Evaluierung	22
8	Quellcode	23
8.1	Gemeinsame Bestandteile	23
8.2	Implementierung des Approximationsverfahrens	26
8.3	Implementierung des Verfahrens fuer optimale Loesungswege	29

1. Einleitung

Dieses Dokument beinhaltet meine Dokumentation der zweiten Aufgabe¹ der zweiten Runde des 43. Bundeswettbewerbs Informatik² in den Jahren 2024 und 2025.

Die vorliegende Problemstellung soll im Folgenden kurz mathematisch modelliert werden. Daraufhin werden wir zunächst untersuchen, wann überhaupt eine Lösung für eine gegebene Eingabe existiert und ein einfaches Approximationsverfahren angeben, welches mit einer schnellen Laufzeit eine nicht notwendigerweise optimale Lösung konstruiert. Anschließend wird eine langsamere Lösung vorgestellt, welche optimale Ergebnisse erzielt. Diese exakte Lösung wird daraufhin mit einer einfachen Methode bezüglich der praktischen Laufzeit (und des Speicherbedarfs) optimiert. Für beide Verfahren wird dabei auch die Platz- und Laufzeitkomplexität analysiert. Die vorgestellten Verfahren lösen jeweils direkt beide Aufgabenteile, also insbesondere Labyrinth mit Löchern. Daraufhin folgt eine Erweiterung der Aufgabe, die darin besteht eine beliebige Anzahl $k \geq 2$ an Labyrinthe zu betrachten. Dieses Problem wird basierend auf den Lösungsverfahren für $k = 2$ gelöst und bezüglich seiner Komplexität untersucht. Daraufhin wird die Implementierung der vorgestellten Lösungsverfahren erläutert. Am Ende folgt die Evaluierung der verschiedenen Lösungsansätze anhand der Beispiele der BwInf-Website. Dabei wird insbesondere die tatsächliche Laufzeit und die Anzahl der Anweisungen der verschiedenen Lösungsverfahren betrachtet. Zuletzt werden die wichtigsten Teile des Quellcodes angefügt und erläutert.

Die vorgestellten Algorithmen basieren auf der Idee, das gegebene Problem als Graphen zu modellieren und anschließend bekannte Ansätze und Algorithmen auf diese Graphen anzuwenden. Genauer, wird eine Bewegungsfolge als Lösungsweg bezeichnet werden, wenn sie in beiden Labyrinthen nach der vorgegebenen Art und Weise vom Start zum Ziel führt. Dann wird zunächst gezeigt, dass es möglich ist einen solchen Lösungsweg zu finden, genau dann, wenn in beiden Labyrinthen ein Weg (also eine Folge von Anweisungen) existiert, der vom Start zum Ziel führt. Dazu wird ausgenutzt, dass Wege in einem Labyrinth umkehrbar und die Person, die als erstes am Ziel ankommt dort stehen bleibt. Diese Konstruktion wird auch genutzt, um eine erste einfache Approximationslösung zu finden, die in Zeit- und Platzkomplexität linear in der Größe der Labyrinth ist (also $\mathcal{O}(nm)$). Dieser Algorithmus betrachtet die Labyrinth als gerichtete Graphen und sucht zunächst in einem Labyrinth nach dem kürzesten Weg vom Start zum Ziel. Die dadurch entstehende Anweisungsfolge wird dann auf das zweite Labyrinth angewendet, sodass ein anderes Startfeld verwendet wird. Ausgehend von diesem Startfeld wird dann im zweiten Labyrinth wieder nach dem kürzesten Weg zum Zielfeld gesucht.

Daraufhin wird der optimale Algorithmus vorgestellt. Dieser Algorithmus sucht nach einem kürzesten Weg auf dem Graphen, der aus allen $(nm)^2$ Zuständen besteht, die die beiden Labyrinthe zusammen erzeugen. Dieser Graph ist anders, als bei den einzelnen Labyrinthen gerichtet, weil es nicht möglich ist, aus einem Zustand wieder in den vorherigen zu kommen, wenn in einem Labyrinth gegen eine Wand gelaufen wurde. Sucht man nun in diesem Graphen nach einem bestimmten kürzesten Weg, so findet man die kürzeste Anweisungsfolge in einer Platz- und Zeitkomplexität von $\mathcal{O}((nm)^2)$. Dieses optimale Verfahren wird anschließend durch eine einfache Methode bezüglich der praktischen Laufzeit verbessert, dabei wird der Graph nicht explizit berechnet, sondern es werden nur die benötigten Kanten berechnet.

Bei der Erweiterung des Problems mit $k \geq 2$ vielen Labyrinth werden zunächst die Lösungsverfahren für den Fall $k = 2$ verallgemeinert. Dabei werden im Approximationsverfahren k kürzeste Pfade zusammengefügt und im optimalen Verfahren wird ein Graph der $(nm)^k$ Zustände betrachtet. Letzteres Verfahren resultiert in einer Platz- und Laufzeitkomplexität von $\mathcal{O}((nm)^k) = \mathcal{O}(2^{\log(nm)^k})$, also einer exponentiellen Laufzeit. Dementsprechend gilt es zu untersuchen, ob dieses Problem überhaupt polynomiell lösbar ist. Diese Frage wird dadurch beantwortet, dass wir zeigen, dass das Problem mit k Labyrinth NP-schwer ist. Dazu wird eine Reduktion des 3-SAT Problems auf das gerade vorgestellte Problem konstruiert. Dementsprechend kann man davon ausgehen, dass es keinen polynomiellen Lösungsalgorithmus, sofern $P \neq NP$.

¹Siehe https://bwinf.de/fileadmin/wettbewerbe/bundeswettbewerb/43/2_runde/Aufgaben432.pdf

²Siehe <https://bwinf.de/bundeswettbewerb/43/>

2. Mathematische Formulierung und Notation

Definition 1: Labyrinth. Wir koennen ein Labyrinth wie folgt vollstaendig beschreiben: Die Menge C seiner Felder und vier Funktionen $l, r, t, b : C \rightarrow C$, die die Uebergange von einem Feld zum naechsten angeben, wenn man nach links, rechts, oben (top) oder unten (bottom) geht.

Bemerkung 2: Dabei ist zu bemerken, dass im Kontext der Aufgabe einige zusaetzliche Eigenschaften erfuellt sind. So ist, zum Beispiel $r(c) = c$, wenn rechts von c eine Wand ist; oder es gilt $r(c) = s_0$, wenn rechts von c keine Wand ist, aber such auf dem Feld rechts von c ein Lock befindet, und s_0 das Startfeld ist.

Auch gilt fuer benachbarte $c, c' \in C$ (die keine Loecher beinhalten) dass $r(c) = c \iff l(c') = c'$ (horizontal benachbart) bzw. $t(c) = c \iff b(c') = c'$ (vertikale benachbart), weil zwischen den Feldern entweder eine Wand ist, oder nicht. In dem Fall, dass keine Wand dort ist, gilt $r(c) = c'$ und $l(c') = c$, bzw. $t(c) = c'$ und $b(c') = c$.

Definition 3: Bewegungsfolge und Wege. Wir betrachten ein Labyrinth, wie oben definiert. Eine Folge $(a_1, \dots, a_k)$ mit $k \in \mathbb{N}$ und $a_i \in \{l, r, t, b\}$ nennen wir Bewegungsfolge. Insbesondere kann man eine Bewegungsfolge auf ein Anfangsfeld $c \in C$ anwenden:

$$(a_k \circ \dots \circ a_1)(c) =: a$$

Wir sagen, dass ein Weg zwischen zwei Feldern $a, b \in C$ existiert, wenn es eine Bewegungsfolge $(a_1, \dots, a_k)$ gibt mit

$$(a_k \circ \dots \circ a_1)(a) = b$$

Fuer eine Bewegungsfolge $g = (a_1, \dots, a_k)$ schreiben wir auch

$$g(a) := (a_k \circ \dots \circ a_1)(a) = b$$

Die unendliche Menge aller Bewegungsfolgen werden wir im Folgenden mit Ω bezeichnen.

Bemerkung 4: Eine solche Bewegungsfolge laesst sich umkehren, wenn in keinem Schritt gegen eine Wand gelaufen wurde und in keinem Schritt ein Lock betreten wurde. In diesem Fall ergibt die Bewegungsfolge $(a_k^{-1}, \dots, a_1^{-1})$ mit $a_i^{-1} = r$, wenn $a_i = l$, usw. (es wird also je l und r , bzw. t und b getauscht). Dann gilt

$$(a_1^{-1} \circ \dots \circ a_k^{-1})(b) = a$$

Dies laesst sich wie folgt formalisieren:

Lemma 5: *Fuer $a, b \in C$ existiert ein Weg zwischen a und b ohne Loecher, genau dann wenn ein Weg zwischen b und a ohne Loecher existiert.*

Beweis: Wegen der Symetrie genuegt es eine Richtung zu zeigen: Es sei $(a_1, \dots, a_k)$ eine Bewegungsfolge mit

$$(a_k \circ \dots \circ a_1)(a) = b$$

O.B.d.A. laeuft man dabei nicht gegen eine Wand, ansonsten entfernt man die entspraechenden Schritte aus der Folge. Dann kann man, wie oben erklart, eine Bewegungsfolge $(a_k^{-1}, \dots, a_1^{-1})$ konstruieren, mit

$$(a_1^{-1} \circ \dots \circ a_k^{-1})(b) = a$$

Definition 6: Problem-I. Wie beschreiben die beiden Labyrinth, wie oben mit der gemeinsamen Menge C der einzelnen $n \cdot m$ Felder, sowie den zwei Anfangs- und Endfeldern $s_0 \in C$ und $t_0 \in C$ und je vier Funktionen l_1, r_1, t_1 und b_1 , bzw. l_2, r_2, t_2 und b_2 . Dabei muss beachtet werden, dass eine Person stehen bleibt, wenn sie das Ziel erreicht. Also gilt $f(t_0) = t_0$ fuer alle Funktionen $f = l_1, r_1, t_1, b_1, l_2, r_2, t_2, b_2$.

Nun soll eine Folge von Anweisungen $(a_1, \dots, a_k)$ mit $k \in \mathbb{N}$ und $a_i \in \{l, r, t, b\}$ gefunden werden, sodass

$$((a_k)_1 \circ \dots \circ (a_1)_1)(s_0) = t_0 \text{ und } ((a_k)_2 \circ \dots \circ (a_1)_2)(s_0) = t_0.$$

Dabei meint $(a_i)_j$ eine der acht Funktionen $l_1, r_1, t_1, b_1, l_2, r_2, t_2, b_2$. Eine solche Folge nennen wir im Folgenden Loesungsweg. In der Aufgabenstellung ist also ein Loesungs-Weg mit minimalem k gesucht. Einen solchen nennen wir minimalen Loesungsweg.

Bemerkung 7: Im Folgenden wird die Unterscheidung zwischen l_1, r_1, t_1, b_1 und l_2, r_2, t_2, b_2 nicht weiter vorgenommen. Dann wird explizit erwahnt, welches Labyrinth grade gemeint ist, oder es ergibt sich aus dem Kontext.

3. Existenz einer Loesung und Approximationsverfahren

Nun soll zunaechst untersucht werden, wie sich charakterisieren laesst, ob ueberhaupt irgendein Loesungsweg existiert. Darauf basierend wird ein erstes Loesungsverfahren angegeben, welches anschliessend hinsichtlich seiner Platz- und Laufzeitkomplexitaet analysiert wird. Dieses erste Loesungsverfahren ist ein einfaches Approximationsverfahren, welches eine nicht notwendigerweise minimale Loesungswege generiert.

3.1. Existenz eines Loesungsweges

Lemma 8: Existenz eines Loesungswegs. *Es existiert genau dann ein Loesungsweg, wenn in beiden Labyrinth ein Weg von s_0 nach t_0 existiert.*

Beweis: Zunaechst bemerkt man, dass ein solcher Weg stets ohne Loecher auskommt. Fuer die Hinrichtung betrachte man, dass der Loesungsweg $(a_1, \dots, a_k)$ offensichtlich einen Weg von s_0 nach t_0 in beiden Labyrinth ergibt (wenngleich eventuell redundante Schritte vorkommen, in welchem man gegen eine Wand laeuft). Die Rueckrichtung ergibt sich aus dem zusammensetzen von zwei einzelnen Wegen innerhalb der Labyrinth: Es gelte $((a_k) \circ \dots \circ (a_1))(s_0) = t_0$ fuer eine Bewegungsfolge $a = (a_1, \dots, a_k)$ im ersten Labyrinth, und $((b_l) \circ \dots \circ (b_1))(s_0) = t_0$ fuer eine Bewegungsfolge $b = (b_1, \dots, b_k)$ im zweiten Labyrinth. Dann sei $((a_k) \circ \dots \circ (a_1))(s_0) =: c$, wobei hier die Bewegungsfolge des ersten Labyrinths auf das zweite angewendet wird. Dann existiert also ein Weg von s_0 nach c im zweiten Labyrinth. Nach einem obigen Lemma existiert dann auch ein Weg zwischen c und s_0 im zweiten Labyrinth, also eine Bewegungsfolge g mit $g(c) = s_0$ im zweiten Labyrinth. Insgesamt gilt dann also

$$(b \circ g \circ a)(s_0) = (b \circ g)(a(s_0)) = (b \circ g)(a(s_0)) = (b \circ g)(c) = b(g(c)) = b(s_0) = t_0$$

im zweiten Labyrinth. Auch gilt $(b \circ g \circ a)(s_0) = (b \circ g)(a(s_0)) = (b \circ g)(t_0) = t_0$ im ersten Labyrinth, weil insbesondere ausgehend von t_0 kein anderes Feld mehr erreicht wird. Also liefert die Bewegungsfolge $b \circ g \circ a$ einen Loesungsweg.

Basierend auf diesem Lemma laesst sich ein einfacher Algorithmus angeben, welcher die Existenz eines Loesungswegs untersucht. Dafuer betrachten wir ein Labyrinth zunaechst als gerichteten Graphen:

Definition 9: Labyrinth als Graph. Wir definieren die Knotenmenge V durch $V := C$ und die Kantenmenge $E \subset C^2$ wie folgt: Zwischen zwei Knoten ist eine gerichtete Kante, wenn sie durch eine der Uebergangsfunktionen voneinander erreichbar sind. Die Menge E der Kanten enthaelt also alle $(c_0, c_1) \in C^2$ mit $c_0 \neq c_1$ und $f(c_0) = c_1$, fuer $f = l, r, t, b$. Diese Kanten sind gerichtet, weil man im Fall eines Lochs nicht immer zurueckgehen kann. Hier soll es keine Selbstkanten (also Kanten von einem Knoten zu sich selbst) geben, insbesondere dann nicht, wenn man dafuer gegen eine Wand laufen koennte.

Nun kann man offensichtlich jeden Weg zwischen zwei Felder eines Labyrinths mit einem Weg zwischen zwei Knoten des grade konstruierten Graphen assoziieren. Dadurch laesst sich durch eine einfache Breitensuche auf dem Graphen bestimmen, ob die beiden Felder s_0 und t_0 im Graphen, also auch im Labyrinth durch einen Weg verbunden sind. Ist dies fuer beide Labyrinth der Fall, so existiert eine Loesung. Andernfalls existiert keine Loesung. Somit laesst es sich mit zwei einfachen Breitensuchen (engl. breadth-first search, BFS) pruefen, ob es einen Loesungsweg gibt:

Algorithm 1: Breadth-first search

Input : Graph $G = (C, E)$, Startknoten $s \in C$, Endknoten $t \in C$
Output: Ob s und t in G voneinander erreichbar sind

```

1  $Q \leftarrow \emptyset$ ,  $Q.push(s)$  // Warteschlange
2  $m \leftarrow \emptyset$  // Liste besuchter Knoten
3 while NOT  $Q$  ist leer do
4    $v \leftarrow Q.top()$ ,  $Q.pop()$ 
5   if NOT  $m$  enthaelt  $v$  then
6      $m.add(v)$ 
7     foreach  $\{v, u\}$  in  $E$  do
8        $Q.push(u)$ 
9     end foreach
10  end if
11 end while
12 return  $m$  enthaelt  $t$ 

```

3.2. Approximationsverfahren - Verbindung zweier optimaler Wege

Mit Hilfe des Verfahrens aus dem Beweis des vorhergehenden Lemmas, zwei Bewegungsfolgen zusammenzufuegen, laesst sich bereits ein erster Loesungsweg konstruieren. Wir betrachten wieder den oben definierten Graphen fuer beide Labyrinth.

Wir setzen nun voraus, dass es einen Loesungsweg gibt. Analog zum Vorgehen im Beweis des obigen Lemmas wollen wir zunaechst einen kuerzesten Weg g von s_0 nach t_0 im ersten Labyrinth finden. Dieser besteht also aus einer Folge von Anweisungen (r, l, t oder b). Nun koennen wir diesen Weg in Labyrinth 2 auf s_0 anwenden. Also $a := g(s_0)$ (in Labyrinth 2). Dann kann man einen kuerzesten Weg h von a nach t_0 im zweiten Labyrinth finden. Das ist moeglich, weil es nach Voraussetzung mindestens einen Weg gibt, naemlich den, der zuerst g rueckwaerts geht und dann den existierenden Weg von s_0 nach t_0 . Nun ist $h \circ g$ ein Loesungsweg. Und somit insbesondere eine obere Schranke fuer die Laenge des minimalen Loesungswegs. Beschreibt man die Zustaende in den beiden Labyrinth durch (c_0, c_1) , wenn Labyrinth 1 in Feld c_0 und Labyrinth 2 in Feld c_1 ist, so erhaelt man folgendes Diagramm:

$$(s_0, s_0) \longrightarrow (g(s_0), g(s_0)) = (t_0, a) \longrightarrow (h(t_0), h(a)) = (t_0, t_0)$$

Hier ist $s_0 \longrightarrow t_0$ minimal in Labyrinth 1 und $a \longrightarrow t_0$ minimal in Labyrinth 2. Tauscht man die Reihenfolge der beiden Labyrinth, so kann man dadurch eine andere Loesung erhalten.

Algorithm 2: Approximationsverfahren

Input : Graphen $G_1 = (C, E_1)$, $G_2 = (C, E_2)$, Startknoten $s_0 \in C$ und Endknoten $t_0 \in C$
Output: Ein Loesungsweg

```

1  $g \leftarrow \text{shortest\_path}(G_1, s_0, t_0)$ 
2  $a \leftarrow \text{apply}(G_2, g, s_0)$  // Wende  $g$  auf  $s_0$  in Labyrinth 2 an
3  $h \leftarrow \text{shortest\_path}(G_2, a, t_0)$ 
4 return  $h \circ g$ 

```

Die kuerzesten Wege erhaelt man nun, indem man in dem Graphen einen Shortest-Path-Algorithmus, wie den Dijkstra-Algorithmus benutzt. Da es sich allerdings um einen ungewichteten Graphen handelt, laesst sich auch eine einfache Breitensuche verwenden:

Algorithm 3: Breadth-first search

Input : Graph $G = (C, E)$, Startknoten $s \in C$ und Endknoten $t \in C$
Output: Vorgaengerarray und Distanzarray

```

1  $Q \leftarrow \emptyset$ ,  $Q.push(s)$  // Warteschlange
2  $m \leftarrow \emptyset$  // Liste besuchter Knoten
3  $a \leftarrow \emptyset$ ,  $a[s] = \text{null}$  // Fuer jeden Knoten, seinen Vorgaenger
4  $d \leftarrow \emptyset$ ,  $d[s] = \infty$  // Fuer jeden Knoten, seine Distanz zum Startknoten
5 while NOT  $Q$  ist leer do
6    $v \leftarrow Q.top()$ ,  $Q.pop()$ 
7   if  $v = t$  then
8     break
9   end if
10  if NOT  $m$  enthaelt  $v$  then
11     $m.add(v)$ 
12    foreach  $\{v, u\}$  in  $E$  do
13       $a[u] = v$ 
14       $d[u] = d[v] + 1$ 
15       $Q.push(u)$ 
16    end foreach
17  end if
18 end while
19 return  $(a, d)$ 

```

Hier wird eine einfache Breitensuche durchgefuehrt (vergleiche zu oben), wobei zusaetzlich zwei Datenstrukturen verwendet werden, die fuer jeden Knoten die Distanz zum Startknoten (also die Laenge des kuerzesten Weges zum Startknoten) - Datenstruktur d - und den Vorgaenger Knoten speichern - Datenstruktur a . Dabei wird in d stets tatsaechlich die minimale Distanz zum Startknoten gespeichert, weil die Knoten beim BFS Stufenweise durchgegangen werden. Das heisst, es werden zunaechst alle Knoten betrachtet, die Abstand 1 zum Startknoten haben, dann alle Knoten, die Abstand 2 zum Startknoten haben, usw. Dementspraechend ist der erste Besuch eines Knotens immer mit der minimalen Distanz zum Startknoten. Das Speichern der Vorgaenger wird wiederum benutzt, um den kuerzesten Pfad nach Ausfuehrung der BFS zu konstruieren. Dazu konstruiert man den Pfad rueckwaerts, indem man mit dem Endknoten $t_0 = t$ beginnt, fuer diesen den Vorgaenger $t_1 = a[t_0]$ bestimmt, usw. bis man den Startknoten erhaelt.

3.3. Analyse der Platz- und Zeitkomplexitaet der Algorithmen

Sowohl die Laufzeit als auch die Platzkomplexitaet sind $\mathcal{O}(nm)$, weil die Graphen der Labyrinth jeweils nm Knoten haben, und jeder dieser Knoten in einem BFS genau einmal besucht wird.

4. Verfahren fuer optimale Loesungen

Nachdem ein Algorithmus, der prueft, ob eine Eingabe des Problems ueberhaupt einen Loesungsweg besitzt und ein erstes Approximationsverfahren angegeben wurden, wollen wir nun ein Verfahren vorstellen, das eine schlechtere Laufzeit aufweist, aber einen minimalen Loesungsweg generiert. Dieses Verfahren wird anschliessend bezueglich der praktischen Laufzeit optimiert. Das Verfahren fuer optimale Loesungen wird daraufhin bezueglich seiner Platz- und Laufzeitkomplexitaet analysiert.

4.1. Loesungsverfahren fuer optimale Loesungswege

Fuer das optimale Verfahren betrachte man die Menge aller moeglichen Zustaende in denen man in beiden Labyrinth zusammen sein kann, also $C \times C$. Fuer einen Zustand $(c_1, c_2) \in C \times C$ gibt es maximal vier verschiedene Folgezustaende, abhaenig davon in welche der vier Richtungen man weiter geht:

$$(l_1(c_1), l_2(c_2)), (r_1(c_1), r_2(c_2)), (t_1(c_1), t_2(c_2)), (b_1(c_1), b_2(c_2)).$$

Dadurch ergibt sich ein gerichteter, ungewichteter Graph auf der Menge der Zustaende $C \times C$, mit maximal $4|C \times C| = 4|C|^2$ Kanten. Wir wollen zunaechst einige elementare Eigenschaften dieses Graphen betrachten:

Lemma 10: Es koennen Schleifen existieren. Zum Beispiel, wenn c_1 und c_2 rechts eine Wand haben.

Lemma 11: Die Kanten sind nicht ungerichtet. Allein schon aufgrund der Existenz von Loechern auf Feldern, die nicht mit s_0 verbunden sind. Aber auch, wenn man in einem Labyrinth gegen eine Wand laeuft, aber auf der gegenueberliegenden Seite der Wand keine Wand ist.

Nun wollen wir die Beziehungen zwischen den zwei Labyrinth und Bewegungsfolgen mit dem Graphen ausdruecken:

Lemma 12: Wege im Graph und Bewegungsfolgen. *Man kann Bewegungsfolgen zusammen mit zwei Startfeldern fuer je ein Labyrinth mit Wegen im Graphen assoziieren, sodass die Laenge der Bewegungsfolge genau der Laenge des Weges im Graphen minus 1 entspricht.* Denn offensichtlich laesst sich aus jedem Weg im Graphen eine entspraechende Bewegungsfolge, und Anfangsfelder konstruieren; und es laesst sich sofort fuer jede Bewegungsfolge mit zwei Startfeldern ein entspraechender Weg im Graphen finden. Und bei dieser kannoischen Abbildung ist der Weg im Graph immer um 1 kuerzer als die Bewegungsfolge.

Das letzte Lemma sagt aus, dass wir jede Bewegungsfolge g zusammen mit einem Anfangspunkt (c_1, c_2) eindeutig mit einem Weg im Graphen assoziieren koennen, sodass der Knoten am Ende des Weges genau der Knoten der beiden Felder $g(c_1)$ und $g(c_2)$ ist. Weiter ist diese Zuordnung insbesondere auch Umkehrbar. Das heisst, dass man fuer jeden Weg im Graphen die dazugehoerigen Anfangsfelder in beiden Labyrinth, sowie die dazugehoerige Bewegungsfolge findet.

Mit dem Lemma laesst sich sofort das schlieszen, dass man anstatt einer minimalen Bewegungsfolge auch equivalent einen kuerzesten Weg im Graphen zwischen den Knoten (s_0, s_0) und (t_0, t_0) finden kann. Welcher dann wiederum einer minimalen Bewegungsfolge entspricht. Um einen solchen kuerzesten Weg zu finden, laesst sich das oben beschriebene Verfahren benutzen, da es sich auch hier um einen ungewichteten Graphen handelt.

Zuletzt wollen wir deswegen nur noch erklaren, wie man den Loesungsweg aus einem kuerzesten Weg des Graphen konstruieren kann. Dies kann man durch iterativ anstellen, indem man fuer jeden Knotenpaar aufeinanderfolgender Knoten im Weg ausprobiert, mit welcher Richtung man vom ersten zum zweiten Knoten kommt.

4.2. Optimierungen des Loesungsverfahrens

Die wichtigste Optimierung basiert auf der Beobachtung, dass in der Breitensuche jeder Knoten nur einmal besucht wird. Genauer, wird der Uebergang zu den Nachbarn nur dann durchgefuehrt, wenn der aktuelle Knoten noch unbesucht ist. Deswegen wird die Menge der Nachbarn jedes Knotens nur einmal benoetigt. Dementspraechend genuegt es diese Menge erst dann zu berechnen, wenn sie wirklich benoetigt wird. Somit muss man den Graphen nicht in einer Datenstruktur speichern, sondern kann ihn implizit darstellen.

4.3. Analyse der Platz- und Zeitkomplexitaet der Loesungsverfahren

Die Analyse der Platz- und Zeitkomplexitaet des normalen Loesungsverfahrens fuer optimalen Loesungen ergibt sich sofort aus der groesze des Graphen. Denn dieser Graph muss in einer Datenstruktur gespeichert werden und die Breitensuche hat die Laufzeit $\mathcal{O}(|V| + |E|)$, wenn V und E , die Menge der Knoten, bzw. Kanten des Graphen sind. Der Graph nun genau $|C|^2 = (nm)^2$ Knoten (weil die Menge der Felder der beiden Labyrinthen natuerlich genau nm ist) und maximal $4|C|^2 = 4(nm)^2$ Kanten. Damit ergibt sich fuer die Platz- und Zeitkomplexitaet jeweils $\mathcal{O}((nm)^2)$.

5. Erweiterung I - k Labyrinth

Nun wollen wir die vermutlich natuerlichste Erweiterung der urspruenglichen Aufgabe behandeln. Statt nur 2 Labyrinth, betrachten wir nun k Labyrinth, wobei eine moeglichst kurze Folge an Anweisungen gefunden werden soll, sodass man in jedem der Labyrinth von einem (fuer alle Labyrinth gleichen) Startfeld zu einem (ebenfalls fuer alle Labyrinth gleichen) Endfeld gelangt. Wir werden zunaechst die Loesungsverfahren fuer den Fall $k = 2$ verallgemeinern, um dieses Problem mit der Laufzeit $\mathcal{O}((nm)^k) = \mathcal{O}(2^{\log(nm)k})$ optimal zu loesen. Somit existiert eine Loesung mit exponentieller Laufzeit (sowie Platzkomplexitaet). Daraufhin beweisen wir, dass dieses Problem tatsaechlich NP-schwer ist, indem wir 3-SAT auf unser problem reduzieren. Somit existiert vermutlich kein polynomieller Algorithmus fuer dieses Problem.

5.1. Verallgemeinerung des Approximationsverfahrens

Wir koennen das bekannte Approximationsverfahren auf k Labyrinth verallgemeinern:

Dazu gehen wir iterativ durch alle Labyrinth und fuegen die jeweils aktuell kuerzeste Bewegungsfolge, die in diesem Labyrinth ans Ziel fuerht zur Folge hinzu.

Algorithm 4:

Input : k Labyrinth und s_0 und t_0

Output: Bewegungsfolge g

```

1 Seien  $c_1, \dots, c_k$  die Positionen in den einzelnen Labyrinth
2  $c_l \leftarrow s_0$  fuer alle  $l$ 
3  $g \leftarrow \emptyset$  // Bewegungsfolge
4 for  $l = 1$  to  $k$  do
5   Sei  $h_l$  die kuerzeste Bewegungsfolge mit  $h(c_l) = t_0$  in Labyrinth  $k$ 
6    $c_l \leftarrow h_l(c_l)$  fuer alle  $l$ 
7   Fuege  $h_l$  zu  $g$  hinzu
8 end for
9 return  $g$ 
```

5.2. Verallgemeinerung des Verfahrens fuer optimale Loesungswege

Auch der Verfahren fuer optimale Loesungswege laesst sich verallgemeinern. Dazu betrachte man den Graphen, der als Knoten C^k hat und fuer $(c_1, \dots, c_k)$ maximal vier ausgehende Kanten nach $(f(c_1), \dots, f(c_k))$ fuer $f = t, r, b, l$. Analog zum Fall $k = 2$ ergibt ein kuerzester Weg von $(s_0, \dots, s_0)$ nach $(t_0, \dots, t_0)$ in diesem Graphen eine minimale Bewegungsfolge, die in allen Labyrinth vom Start zum Ziel fuehrt.

5.3. Beweis der NP-Schwere des Problems mit k Labyrinth

Nun wollen wir die NP-Schwere unseres Problems mit k Labyrinth beweisen. Dazu fuehren wir eine Reduktion von Exakt-3-SAT auf unser Problem durch. Denn Exakt-3-SAT ist ein bekanntes NP-vollstaendiges Problem. Genauer reduzieren wir Exakt-3-SAT auf die Entscheidungsvariante unseres Optimierungsproblems, welches die Entscheidungsfrage stellt, ob es einen Loesungsweg fuer alle k Labyrinth gibt, der maximal $L \in \mathbb{N}$ Anweisungen enthaelt. Mit Exakt-3-SAT meinen wir die Variante von 3-SAT in welcher jede Klausel der gegebenen Formel genau 3 verschiedene Literale enthaelt.

Wir betrachten nun also eine Eingabe des 3-SAT Problems, bestehend aus l Variablen $x_1, x_2, \dots, x_l$ und einer Formel F in konjunktiver Normalform mit p Klauseln, und wollen sie in eine Eingabe unseres Problems umwandeln, bestehend aus k Labyrinth mit je der Groesze $n \times m$ und einer Zahl $L \in \mathbb{N}$, sodass gilt

Es existiert eine Belegung der l Variablen, sodass F wahr wird
 $\iff$ Es existiert eine Bewegungsfolge g mit $|g| \leq L$ und $g(s_0) = t_0$ in allen Labyrinth.

Fuer die Eingabe des 3-SAT Problems nehmen wir an, dass fuer jede der p Klauseln genau 3 Literale hat und jedes davon verschieden ist. Ausserdem nehmen wir an, dass niemals x_i und $\neg x_i$ in einer Klausel vorkommen. Denn in diesem Fall ist die Klausel unabhengig von der Belegung immer wahr, sodass man sie auch weglassen kann. Zuletzt nehmen wir an, dass jede Variable in mindestens einer Klausel vorkommt, denn ansonsten koenne man sie auch weglassen.

Um nun die Eingabe unseres Problems zu konstruieren werden wir fuer jede der p Klauseln ein Labyrinth konstruieren, also $k = p$, wobei jedes Labyrinth die Groesse $(2l + 4) \times 15$ hat. Somit setzen wir die Anzahl der Spalten $n = l + 3$ und die Anzahl der Zeilen $m = 15$.

Wir betrachten nun die Klauseln $C_j = l_{k_1} \vee l_{k_2} \vee l_{k_3}$ fuer $j = 1, 2, \dots, p$, wobei $l_{k_1}, l_{k_2}, l_{k_3}$ irgendwelche Variablen oder ihre Negation sind. Nun seien die Variablen die in der Klausel C_j vorkommen (egal ob negiert oder nicht) $x_{i_1}, x_{i_2}, x_{i_3}$. Dabei sei oBdA. $i_1 < i_2 < i_3$ in jeder Klausel. Die Literale in einer Klausel haben also eine fixe Anordnung. Nun sei $\text{index}_j(x_i)$ die Position (1, 2 oder 3, von links nach rechts) von x_i in der Klausel C_j , bzw. 0, wenn x_i nicht in C_j vorkommt. Damit definieren wir

$$L = 16 + 2 \cdot \sum_{i=1}^l 1 + 2^{3-m_i},$$

mit

$$m_i = \min \left\{ \text{index}_j(x_i) : j \in \{1, 2, \dots, p\} \right\}$$

Das heisst, fuer eine der l Variablen ist m_i der kleinste Index an welchem die Variable x_i (negiert oder nicht negiert) in irgendeiner Klausel steht. Insbesondere ist $1 \leq m_i \leq 3$, weil jede Variable in mind. einer Klausel vorkommt und der maximale Index 3 ist.

Nun wollen wir endlich fuer eine Klausel C_j mit den Variablen (egal ob negiert oder nicht) $x_{i_1}, x_{i_2}, x_{i_3}$, das j -te Labyrinth konstruieren. Rechts von der ersten Spalte setzen wir ueberall Waende ausser in der 8. Zeile (die Zeilen und Spalten sind 1-indexiert). In den l folgenden Spalten assoziieren wir die $(i + 1)$. Spalte mit der i . Variable x_i .

Wir konstruieren nun das Labyrinth. Betrachte das Labyrinth aus der Abbildung. Wir nehmen dieses, schneiden

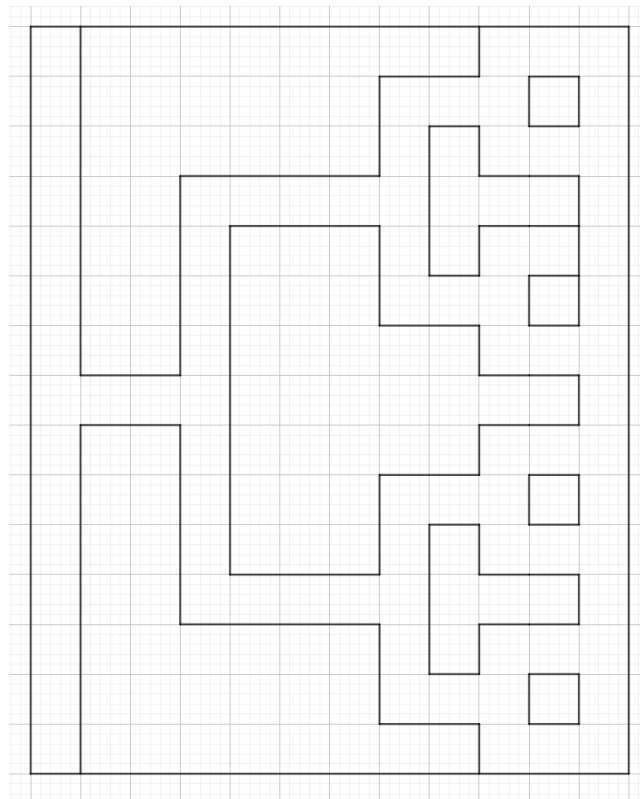


Abbildung 1: Linke Haelfte des Labyrinths zu $(\neg x_3 \vee x_7 \vee \neg x_9)$ fuer $l = 9$

die rechteste Spalte ab, und spiegeln an der neuen rechten Seite. Dadurch erhaelt man ein Labyrinth mit 15 Zeilen und $2l + 4$ Spalten.

Wie man sieht entspricht die linke Haelfte jedes Labyrinths einem Entscheidungsbaum der Tiefe 3, wobei die jeweiligen Entscheidungen (hoch oder runter) in den Zeilen $i_1 + 1, i_2 + 1$ und $i_3 + 1$ getroffen werden. Dabei interpretieren wir hoch gehen, als x_i auf 1 setzen und runter gehen, als x_i auf 0 setzen. Wenn man die $(l + 2)$. Spalte des Labyrinths erreicht hat, hat man also eine Belegung der drei relevanten Variablen getroffen. Nun verschliesst man den Ausgang des Labyrinths mit einer Wand, der die Klausel nicht erfuellen wuerde (s. Zeile

5 des Beispiels). Wir zeigen nun, dass die oben genannte Äquivalenz tatsächlich gilt:

Dazu bemerken wir als erstes, dass es nicht keinen Lösungsweg für alle Labyrinth mit weniger als L Anweisungen geben kann. Wegen der Symmetrie zeigen wir, dass es mindestens $L/2$ Anweisungen braucht um in allen Labyrinth in die $(l+2)$. Spalte zu kommen. Dies liegt daran, dass man zunächst 8 Anweisungen braucht, um in die zweite Spalte zu kommen. Daraufhin existiert für jede Spalte i ein Labyrinth in welchem man mindestens $1 + 2^{3-m_i}$ Anweisungen braucht um in die nächste Spalte zu kommen. Induktiv lässt sich so zeigen, dass es keine Bewegungsfolge gibt, die weniger als $L/2$ Anweisungen braucht, um in die $(l+2)$. Spalte zu kommen. Wegen der Symmetrie folgt daraus, dass man für das gesamte Labyrinth mindestens L Anweisungen braucht, wobei man natürlich alle Anweisungen spielen muss.

Hinrichtung:

Es existiere eine Belegung der Variablen, sodass F wahr wird. Wir konstruieren eine Folge, die die erste Hälfte des Labyrinths zu $(l+2)$. Spalte durchläuft:

Als erstes gehen wir 7 mal nach unten und einmal nach rechts. Dann definieren wir l Teilfolgen für jede Variable x_i , die wir aneinanderreihen werden. Jede der l Teilfolge sieht wie folgt aus: Wird $x_i = 1$ gesetzt, so wollen wir nach oben gehen, und zwar genau 2^{3-m_i} mal, und dann einmal nach rechts gehen. Wenn $x_i = 0$ gesetzt wird, so gehen wir analog 2^{3-m_i} mal nach unten und einmal nach rechts. Per Definition ist diese Folge gerade genau $L/2$ Schritte lang. Wir zeigen nun, dass diese Folge auch für jedes Labyrinth funktioniert.

Wir fixieren das Labyrinth der Klausel C_j . Mit den ersten 8 Schritten gelangt man in Spalte 2. Nutzt man nun die l Teilfolgen nacheinander endet man offensichtlich in der $(l+2)$. Spalte. Aber die Position an der wir nun sind codiert genau eine Belegung der 3 Variablen der Klausel C_j . Aber diese Position muss eine erfüllende sein, wegen der Konstruktion der Folge. Also kann hier keine Wand sein, die den Weg blockiert.

Aus der einen Hälfte erhält man durch Spiegelung eine Bewegungsfolge für die rechte Hälfte des Labyrinths. Die Folge ist dann insgesamt L Anweisungen lang.

Rueckrichtung:

Es existiere eine Bewegungsfolge mit maximale L Anweisungen, die in jedem Labyrinth vom Start zum Ziel führt. Wir zeigen, dass es eine erfüllende Belegung der Variablen gibt:

Dann muss diese Folge genau L Anweisungen haben. Da genau L Anweisungen benötigt worden sind, wissen wir, dass für jede Variable eine Entscheidung (hoch oder runter) getroffen wurde. Denn es gibt keine anderen Wege durch das Labyrinth mit L Anweisungen. Also können wir die entsprechende Entscheidung für jede Variable übernehmen. Weiter wissen wir, dass dadurch alle Klauseln erfüllt sind. Denn ansonsten wäre der jeweilige Weg nicht möglich.

6. Implementierung

Die vorgestellten Loesungen wurden allesamt in C++ implementiert. Die Visualisierung, die im folgenden Kapitel zu finden ist, wurde in Python implementiert und wird hier nicht naeher behandelt.

6.1. Darstellung der Labyrinth

Die Darstellung der Labyrinth wurde in ein struct *Maze* ausgelagert. Dieses speichert die Anzahl der Zeilen und Spalten eines Labyrinths, sowie zwei Datenstrukturen, die Position der Waende, und die Position der Loecher speichern. Die Letztere Datenstruktur ist ein zweidimensionaler *vector*, der Wahrheitswerte fuer jedes Feld speichert, ob dort ein Loch ist oder nicht. Die erste Datenstruktur ist ein array, dass vier (fuer jede Richtung einen) zweidimensionale Vektoren an Wahrheitswerten beinhaltet, welche fuer jedes Feld aussagen, ob sich in der jeweiligen Richtung eine Wand befindet.

Um Positionen in einem Labyrinth zu speichern wird das struct *Coordinate* benutzt, welches einfach zwei Ganzzahlen fuer die Zeile und Spalte beinhaltet und Operationen, wie Addition und Subtraktion unterstuetzt. Diese Datenstruktur wird auch als Knoten in den Graphen der Labyrinth benutzt.

6.2. Weitere Implementierungsdetails

Um die Knoten im Graphen des exakten Verfahren darzustellen, wird ein weiteres struct *Node* benutzt, welches aus zwei *Coordinate* Variablen besteht. Weiter werden Graphen standardmaeszig als Adjazenzlisten gespeichert, bzw. im Fall des exakten Verfahrens garnicht gespeichert.

Ein weiteres Implementierungsdetail besteht in der internen Unterscheidung zwischen den Richtungen. Dafuer wurden Konstante Ganzzahlen *UP*, *RIGHT*, *DOWN* und *LEFT* mit den Zahlen von 0 bis 3 definiert. Dann ist beispielsweise der Eintrag *array[RIGHT]* in *Maze* der zweidimensionale Vector der die rechten Waende beinhaltet. Ausserdem laesst sich so sehr einfach durch alle vier Richtungen iterieren.

7. Evaluierung anhand von Beispielen

Nachdem die Lösungsvorschläge ausführlich erklärt wurden, und auf die Implementierung eingegangen wurde, sollen nun die Ergebnisse der Lösungsverfahren vorgestellt werden. Dabei wollen wir die Ergebnisse für die vorgegebenen Beispieleingaben zunächst zusammenfassen und daraufhin einzeln angeben. Zuletzt wird ein kurzes Fazit gezogen.

7.1. Zusammenfassung der Ergebnisse

Nun sollen zunächst kurz die Ergebnisse aller Beispieleingaben der BwInf-Website zusammengefasst werden. Dazu wird in der folgenden Tabelle für jede der 10 Eingaben zunächst die Größe, sowie die Anzahl der Löcher g_1 , g_2 der Labyrinth angegeben und anschließend die Ergebnisse der beiden Algorithmen, wobei jeweils die Anzahl der Anweisungen und die Laufzeit des Algorithmus in Sekunden (gerundet auf drei Nachkommastellen) angegeben wird. In der letzten Zeile ist (gerundet auf zwei Nachkommastellen) der relative Fehler zwischen der Approximation und der optimalen Lösung in Prozent angegeben. Dabei ist darauf hinzuweisen, dass für die Eingabe labyrinth6.txt wegen des hohen Speicherverbrauchs keine optimale Lösung angegeben werden konnte.

Eingabe	(n, m)	(g_1, g_2)	Approximation		Optimal		Fehler [%]
			Anweisungen	Laufzeit [s]	Anweisungen	Laufzeit [s]	
labyrinth0	(3, 3)	(0, 1)	8	0,000	8	0,000	0,00
labyrinth1	(5, 4)	(0, 0)	32	0,000	31	0,000	03,26
labyrinth2	(10, 10)	(3, 9)	86	0,001	65	0,002	32,31
labyrinth3	(10, 50)	(0, 0)	221	0,003	164	0,053	34,76
labyrinth4	(101, 101)	(0, 0)	20168	0,046	14384	21,760	40,21
labyrinth5	(151, 151)	(3443, 3421)	1742	0,079	1308	139,893	33,18
labyrinth6	(250, 250)	(24302, 24338)	1847	0,173	?	?	?
labyrinth7	(30, 10)	(6, 7)	∞	0,001	/	/	/
labyrinth8	(150, 150)	(0, 0)	716	0,087	472	143,568	51,69
labyrinth9	(170, 170)	(0, 0)	1377	0,128	1012	419,344	36,07

Tabelle 1: Ergebnisse der zwei Verfahren

7.2. Die Beispiele der BwInf-Website

Nun werden die Beispielergebnisse einzeln behandelt. Dabei wird jeweils die Anweisungsfolge der Approximation und der optimalen Lösung angegeben. Eine Anweisungsfolge besteht dabei aus Buchstaben (U, R, D, oder L), die von Leerzeichen getrennt werden. Dabei steht U fuer up (hoch), R fuer right (rechts), D fuer down (runter) und L fuer left (links). In den Ausgabedateien wird zusaetzlich immer noch die Folge an Zustaende angegeben, die durch die Bewegungsfolge entsteht. Ausserdem wird fuer einige Eingaben eine Visualisierung der beiden Labyrinth angegeben, wobei die Waende gruene eingezeichnet sind, und die Loeche als rote Punkte dargestellt werden.

1. Beispiel: labyrinth0.txt, $n = m = 3$

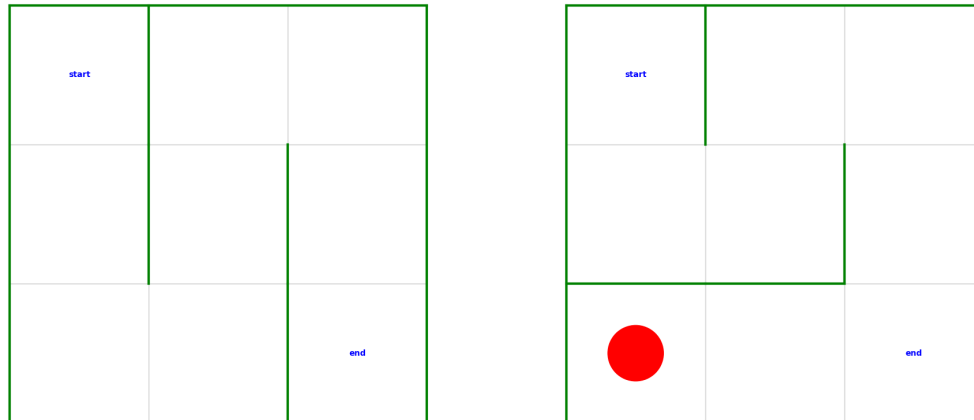


Abbildung 2: Visualisierung der Eingabe labyrinth0.txt

Die Approximation erreicht 8 Anweisungen, waehrend die optimale Loesung 8 Anweisungen erreicht.

Approximation: D D R U U R D D

Optimale Loesung: D D R U U R D D

2. Beispiel: labyrinth1.txt, $n = 5, m = 4$

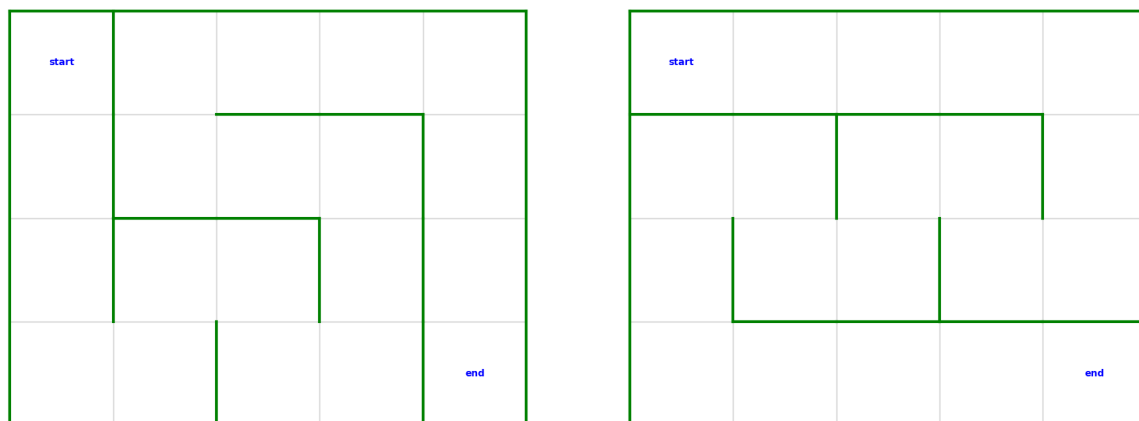


Abbildung 3: Visualisierung der Eingabe labyrinth1.txt

Die Approximation erreicht 32 Anweisungen, waehrend die optimale Loesung 31 Anweisungen erreicht.

Approximation:

DDDRURDRUULLURRRDDDLULDLULDDRRRR

Optimale Loesung:

RRRRDDLULDLULDDRURDRUULLURRRDD

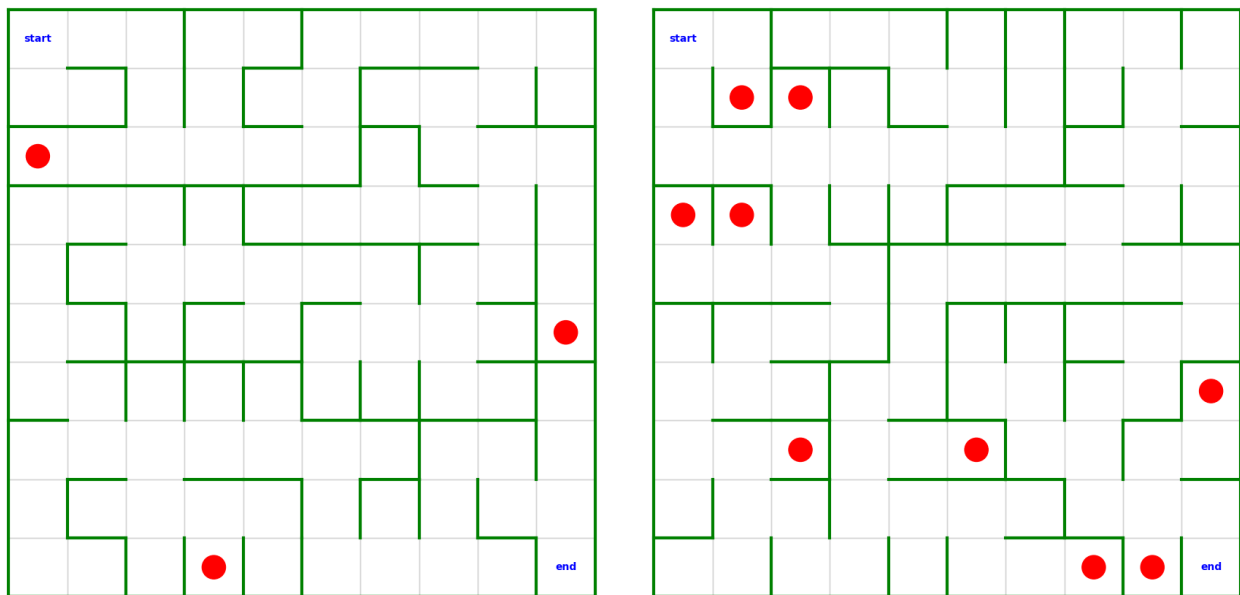
3. Beispiel: labyrinth2.txt, $n = m = 10$ 

Abbildung 4: Visualisierung der Eingabe labyrinth2.txt

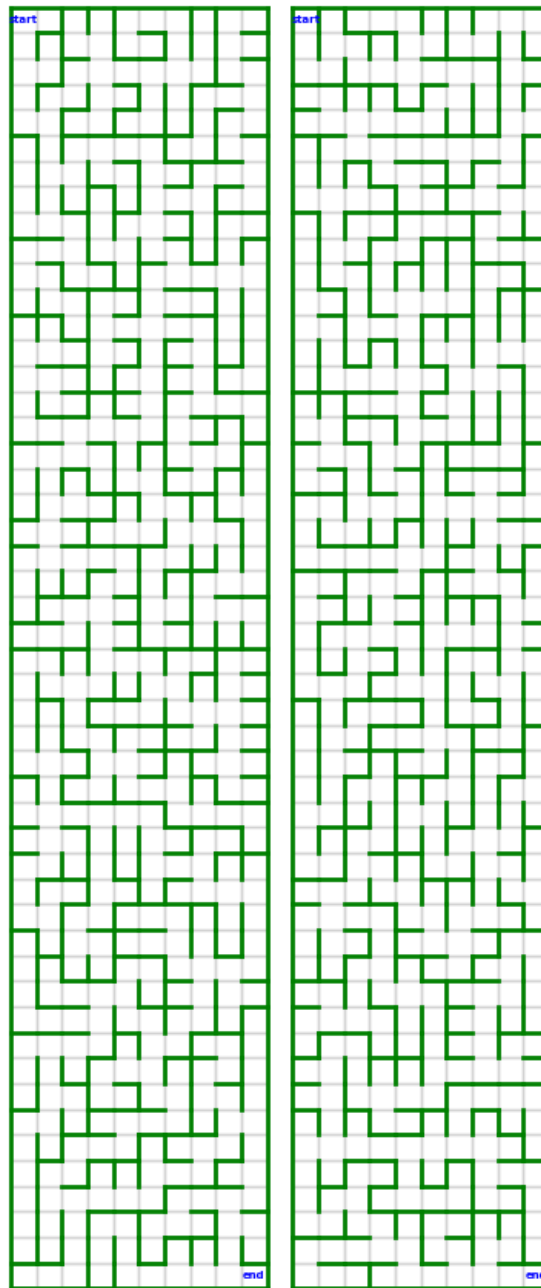
Die Approximation erreicht 86 Anweisungen, während die optimale Lösung 65 Anweisungen erreicht.

Approximation:

```
RRDDRRRUURRRDLDRDDLULULLLULLDDDRDRRRRDDRURURD
RDLLLDDRDLDDLDRDRDRUUURUUURRRRRDLDDLDDR
```

Optimale Lösung:

```
RRDDRRRUURRRDLDRDRDLULULLLULLDDLDRDRRRRDDRURU
UURUUURRRRRDLDDLDDR
```

4. Beispiel: `labyrinth3.txt`, $n = 10, m = 50$ Abbildung 5: Visualisierung der Eingabe `labyrinth3.txt`

Die Approximation erreicht 221 Anweisungen, während die optimale Lösung 164 Anweisungen erreicht.

Approximation:

```

DDDDRDDRURRRDDDDRDLDDDDDDLLDLDDDDRDRRDLDDDD
LLULDDDDRDRRRRRDRDRDRDDDLDDLDLDDRDLDDLDDL
DRRRURURURDDDDLDDDRUULDLDLLDLDDDDDRDDRDRD
DDDLDRDLDDLULDDRDRRDLDDRDDRDRDDLDDDRURDDL
DLULULULDLDRDDRDRDDDDLULUULDDDDLDDRURDRDRDRRRR

```

Optimale Lösung:

```

DDDDRDDRURRRDDDDRDRRDLDDDLDDDLDDLDDRDRDRDRD
DRLDDDLDRDLDDLULDDRDRDRDLDDRDRDRDRDRDDLDD
DLDDLDRURDDLDDLULULULDLDRDRDRDRURURURDDDDL
UURLDDDDLDDRURDRDRDRDRRRR

```

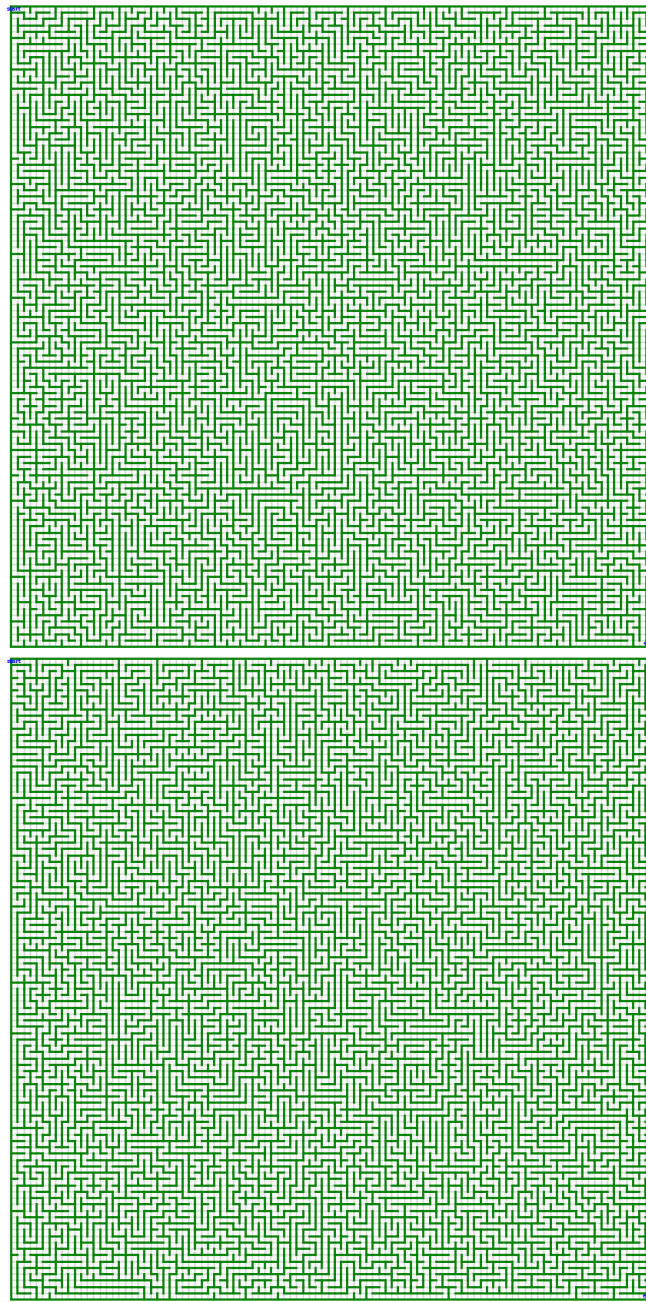
5. Beispiel: labyrinth4.txt, $n = m = 101$ 

Abbildung 6: Visualisierung der Eingabe labyrinth4.txt

Die Approximation erreicht 20168 Anweisungen, während die optimale Lösung 14384 Anweisungen erreicht. Wegen der großen Anzahl an Anweisungen geben wir die die Anweisungsfolgen nicht vollständig an, und verweisen dafür auf die Lösungsdateien.

Approximation:

```
RRDRURRRD LLDLLULDDDRURRRRUURRUURRRRRD LLLLDLDR
RRRULURRDDDDDRDDLULUULDDDLDDRURDDR
```

...

```
RULLURRRRRRULURULULDDDLUULURUURDRURRDLDRDDDDDL
LLDRRRRDLLDRDLDRDLLUULDDLDRRRR
```

Optimale Lösung:

```
RRRDDLULLDRDLDRDLDRRUULURDRURDDLDDLULLULDDD
```

RURRRDDDRUURRULLURRRRUUUUUURDLULD

...

RRRRRDRULURULULDDDLUULURUURDRURRDDLDURURDDDDDL
LLDRRRRDLDDRDLDRDLLUULDDLLDRRRR

6. Beispiel: labyrinth5.txt, $n = m = 151$

Die Approximation erreicht 1742 Anweisungen, waehrend die optimale Loesung 1308 Anweisungen erreicht.

Approximation:

```
DDRRTTRTRRDLDDDRRDLDLLDDLDLDDDDDRDDRRRDRRRRRDL
DDDDDDDDRRRTTTRTTTRRTRRRRTRRTRDRTRRRRDDDDDRDDDDDDDR
RDRDDDDDDL LLLLDLDRDLDRDDDRDRRRRDDDDL D L L D D R D D R R
DRDRDRTRRRRTLLTRRTRRRTLLLT T T T T L T T R D R R R T T L T T T R T T R T
T T T T R R R T T T R R R D D R R R D R D D L D D L D D D L D R D R D R D R D D L
D D L D R R D L D D D R T T R T T R D R R R T R T T T T R R R R T R R D D R R T R T T T T R
R D R D R R R T R R R D D R T T T L T L L T T R T R R R R D R T R T T T T R R D D R T T R R
R R R R D D L L D D R D D R D D D R D L L L L L D D D D L D R R R D D L L L D R R D D D D R
T R R R R R D R R R D D D R R T R R R R D R D R T R R R T T L T T L T T L T R T T R R R R R
T R D R T R D D R D R D D D D L D R R R D D L D L T L D D D D R R R D D D R D R D L L L T
L L D D L L D R D D D R D R T R D R R T T R R R R R R T T L T T L T R R T R D R T R D D R R
R R R R D R R R T R D R D R D D D R T R T T T R T R R T T T R T T R T L T R R D D D R D D L
T L D D L D D R D L D D L D R D R D R D D D R D R R D L D D D D R D L L D L D L T T L L D
L L L L D L L T T T T T L T L T L L L D L L L T T L D L D D L D L D D D D D D R T R D R D D L
L D L D L D L L L T L L D R D L L L L L T L L L L L L L D R R D D L L D D R D R D R D D R
R R D L L D L D L D L L D R D R D D L L D R R D R D D D L L L D L L D D L L L D D D L T
L D D D L L T T L L D D D D R D R D D D L D D R R R D D R D D R T R T R R R R R D R R R D R
R R R T R R D R D R R R R R R T T T L T T L L L T R T T R T T T T R R D R D D R T T R T L T
T R T T L T T R R R R T R T R R T T R R R R D D D D R R D R R R D R R D R R D R R D D L L
L L L D D D D L L T L D D L D L D D D D D R T T R D R R R D R R D D D L L D R D R R R R D
D L D R D D R R R R D D R D D R D L L T L L D D L L D R D R D R R D L D R D R D D L D D
R R D R D R T R D D D L D R R R R D D D D D L L D R R R R R T R D D D D L L D D D D R D D
L D D D D R R T T R T R T T R D R T T T T R R D R D R D R T R D R D R D R D D R R D R T T R
R R R D R R R T T R R R T T T L T R R T T L L L L T T R T T T R R T R D D R T R T R D R T R
R R R D L L L D R D D L L D D R R T R D R D D D L L D D D R T R T R R R D D L D D D R D D
R D D D D R D R R R R D R D R D R D D D D D D D D L L L L D L L D L T L T L L D L L D D D
L L D L D R D L L D D D D D D L L D R D D D L D L L L D L L T L D L L L L L T L D D L D L L L
D L L D D D D D R D L D D L D D R R D D D D D R T T R D R D D D D D L L L L D L T L L T L D
L D D D D R D R T R D R R D L L D R R D D D D R D R D D D R T T T L T R R R R R R T T T R
R R T R R R D R D D R R D R D R T R T L T T R D R R T R R D D D D R D D R R R D R T R R D
D D D L D D D R D D R D R D D L T L L D L L D L T L T T L L L L D D D R T R D R D D L L D L
L L L D R R D L L D L L D R D D D R R R D R R R D R R R T R T T T R R D R D R T T T L L T T
R R R T R T R T R D R R T R T T R R R T T T L T T T T L L T T L L T L L T T R R R D R T R T
T R D D R R R R R T R R R D D L L D R D R D D D D R T R R D D D R T R R T T L T R R D R
R T R R R T R R T R R T T R D R T R R R D R R T R R T L T L T T R R D R R D D R D D R T R
R D D D L D R R R R D D D D D L L D D D R D L D D L D D R R D D R T R R D R T R D R D D
L D L D R R D R D R T T T T R T T R T L T R R T R T R R R D D R T R D R R T R D R R T T T R
R R R R R R D D D D D R D R D R R D R D L D D L L L D D R D L D D D D L D R D R R R R R D D R D
```

Optimale Loesung:

```
DDRURURURRDLDDDRRDLDLLDDLDLDDDDDRDDRRRDRRRRRDL
DDDDDDDDRRRUURUUURRURRRURRURDRURRRRDDDDDRDDDDDDDR
RDRDDDDDDL L L L L D L D R D L L D R D D D R D R R R D D D D L D L L D D R D D R R
DRDRDRURRRULLURRURRULLLUUUUULUURDRRRRUULUUURUUR
UUUUURRRRUURRRRDDRRRDRDDLDLDDDLDRDRDRDRDRRDD
LDDLDRRDLDDDRUURUURDRRRURUUUURRRRRURRDDRRURUUUU
RRDRDRRRURRRDDRUUUULULLUURURRRRRDRURUUUURRDDRUUR
RRRRRRDDLDDRRDDRRDDDRDL L L L L L D D D D L D R R R D D L L L D R R D D D D
RURRRRRDRRRRDDDRURRRRRDRDRURRRRUULUULUULURUURRRR
RURRDRURDDDRDRDDDDLDRRRDDLDLULDDDDDRRRRDDDRDRDLLL
```

```

ULLDDLLDRDDDRDRURDRRUURRRRRRUULUULURRURDRURDDR
RRRRRDRRRURDRDRDDDRURUUURURRUUUURUURULURRDDDRD
DLULDDDLDDRDLDDLDRDRDRRDDRDRRDLDDDDDRDLDDLDDL
LLDLLLLDLLUUUUULULULLLLDLLLUULDLLDDDLDDDDDDDRURDR
DDLDDLDDLDDLULLDRDLULLLLULLLLLLLLDRRDDLLDDRDRDRD
DRRRDLDDLDDLDDLDRDRDDLDRRDRRDDLLLLDDLDDLDDL
ULDDDDLULULLDDDDDRDRDDDDLDDRDRRDDURURRRRRDRRRDR
RRRURRDRDRRRRRRRUUULUULLLURUUURUUURDRDDRURULUU
RUULUURRRRURURRUURRRRDDDDDRDRRRDRRRDRRRDDLLL
LDDDDLULULLDDLDDDDDRUURDRRRDRRDDDLDRDRRRRDDLD
RDDRRRRDDRDRDDLULLDDLDRDRDRRDLDRDRDDLDDRDRDR
DRURDDDLDRRRRDDDDDLDRRRRRURDDDDLLDDDDDRDDLDDDD
RRUURURUUURDRUUURRDRDRURDRDRDRDDRDRUURRRRRDR
RRUURRRUUULURRUULLLLUURUUURRURDDRURURDRURRRRDL
LLDRDDLDDRURDRRDDDLDDRURURRRDDLDDRDDRDDRDDR
DRRRRDRDRDRDDDDDDDDDLLLLDLDDLULULLDDLDDLDDLDRD
LLDDDDDDDLDRRDDDLDDLDDLULLDLULLLLULLDDLDDLDDL
RDLDDLDDRDRDDDDDRUURDRDDDDDLLLLDLULLDLDDDDDRDR
URDRRDLDRRDDDRDRRDDRUUUULURRRRRRUUUURRRURRRDRD
DRRDRDRURULUURDRRURRDDDDDRDDRDRURRDDDDLDDRDRD
DRDRDDLULLDLDDLULUULLLLDDRURDRDDLDDLDDLDRDDL
LLDRDDDRRRDRRRDRRRURUUURRDRDRUUULLUURRRURURURD
RRURUUURRRUUULUUUUULLUULLLURRRDRURUUURDDRDRRRU
RRRDDLLDRDRDDDDDRURRDDDRURRUULURRDRRUURRRURRUR
UURDRURRRDRRUURRULULUURRDRRDDDRDDRURRDDDLDRRRRD
DDDDLDDLDDRDLDDLDDRDRDDRURRDRDRDDLDDLDRRRDRRUU
RUURULURRURURRRRDDURDRRURDRRUUUURRRRRRRRDDDDDR
DRDRDLDDLDDLDDRDLDDDDLDRDRRRRRRRRDDDRD

```

7. Beispiel: labyrinth6.txt, $n = m = 250$

Diese Eingabe ist mit 250×250 die Grösste Eingabe, und in sofern besonders, dass sie keine Waende, sondern nur viele Loecher enthaelt.

Die Approximation erreicht 1847 Anweisungen:

```

RRRRRRRRRRRRRRRRRRDRRRRRURRDDDRDRRRURRRDDRDDLDDRDRR
DRRURURURURURRDDDRDRDRDRRRRUUUURRRDDDDDRRRURRRDRDD
DRRUUUURRDRDRRRRUURRDDDRRRRRRUURRRUULUURRRRRDDDDDD
RRRRDRDRRRRDDLDDDRDRRRRRRDDRDDDLDDDDDRDRDDDDDRDRRU
RRRRRRRRDDRURUUUURRUULUURRRRRRUURRRUURRDDDDDDDDDDR
DDLDDLDDDDDRDDDDDDDRDRDRDRDDDRRUUUUURRRRRDRDDDDDDL
LDDDLDDLDDLDDLDDDDDRRRRRRDDDDDDRRRRRDDDLDDRDRRDDLD
LDLDDLDDLDDLDDLDDDDDRDRDDDRRRRRRRRUURRRDRRRURURURR
RUUURUUUUULUULULUURRRURRUUURRRURRUULLUUUURUUURURU
RUUUURRRRRUULULUURRUUUUURRRDRRRRRRUURRURUULLUUR
URURRRRRURRDDDRRRRDDLDLDLDDRDRRRRUUURRRRRDDLDDL
DRDRRUURURRDDDRRDDDDLDDLDDLDDRDRRDDLDLDDDDDDDRDDL
DDDLDDLDDLDDLDDLDDLDDRDRRRDRDRDRURURRRURRRRR
DDDRRRDRRDDDDDDDLDDLDDRDRRDDDLDDLDDRDRRDD
DDDLDDLDDLULULLDDRDRDDDDDRRRRRRDDDRDRRRRDDDL
LDLDDLDDRDRRRRDDDRRRURRURURRRRRURRDRRRRDDLDLDDL
DRRRDDDLDDLDDLDDLDDLDDDDLLUUULLDDLDDLDDLDDRDRRD
RRURURRDRRDDDDDDDDDRDDLDDRDRRRRRRDDDRDRRRRRRUUU
RRURRRRRDRRRRUURRDRRRDRRRRUURRURRRDRRDDDRDRDDL
LDDDDLDDDDLDDLULLLLLLDDDDLDDLDDLDDLULLLLDDLDDLDRDR
DRDDLDDLULLULULLDDLDDLDDRURRRDRRDDRRDDLDDDDDR
DDDRRRRRRRDRRRRRRUURUUURURRDDDRDDDDDDDRDDRDRRRUU
RRRDRDRURURRURURRRRRDDRDDLDDLDDRDRDDLDDLDDL
RRDRRRDRRUURRRDDDDDDDDDRDRDRDDRDRRRURRDDDRRRD
DDDLDDLDDRDDDDDDDRDDLDDRURRUURRRDRRRRRRRURRDRRU
RURRDRRRRRRRRDDDRRRRRDDRDRRRRRRDDDDLDDLDDRDRDDDD

```

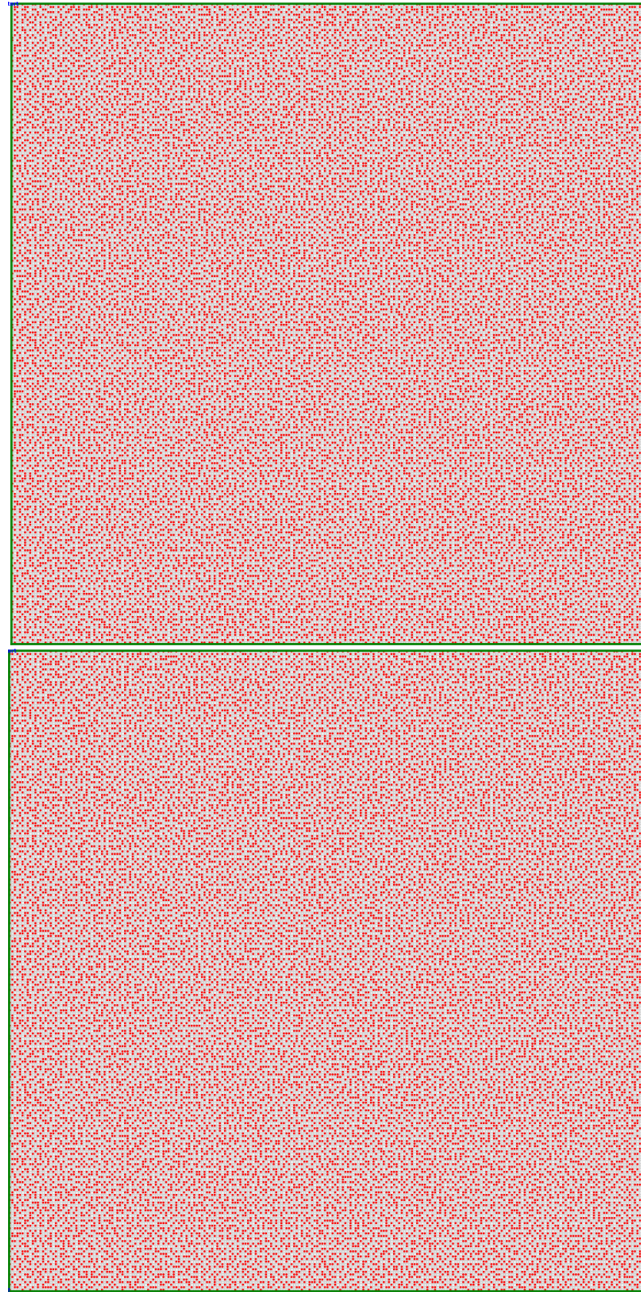


Abbildung 7: Visualisierung der Eingabe labyrinth6.txt

```

R D D R R D R R R R D R R R R D D R R D R R R U U L U U U U R R U U U U R R R R R D R R D R
D R R R D D R R R U R U R U R R U R R R R D R D R D R D D D R R R R D R D D L L L L D D D D
R D D D L D D D R D R D D D D R R R R D R R R D D R R D R D R R R D D D D L D D D D L L L D
D L D D R D R D R R R R R R R R R U U R R R D R D R D R D R R R D D D D D R D R R R R D D
D D D D R R D R D R D D D D R U R R R R R U U R R R U R R D D D D L D D R R U R R D D D D
D R R D D D D D L D D D L D D D D L L L L D D D R D R D D D R R R R D R D D L D D R R D R
R D D R U U R R R R D D D D D D L D D R R R D R R D D R R R U R U R U U U U R U R R U R
R D R R R D D R R D D R R U R U U R R R U R R U R U L U U R R U R R R D D R R R R R R D D
D R D D D D D D D R D R R D D R R U R R D D L D L D L L D L D D R R R U R R D D D R R R R U
U R R R R U R R R R D R D D R R U U R U U U R R D R D R R R D R R D D D L D L D L L L L D
L L D D L D D D L L D L D L L L D L D D R D D L L D D R R R R D D D D D D D D D R D D D R D
D R D D L D D R R D D L D L D D D R R U U R R D D D D R R U R R U U R R R R U U L U U R R D
R R D D D R R R D D D R D R R R R D R R R U U R R R D D D D D R R R D D D D L D D L D D D
D D D R R D D L D L D D D D D L D L D D L D D D L D D D D D D D R R D D D L D L L D D R D D
R R R R U U R R D D

```

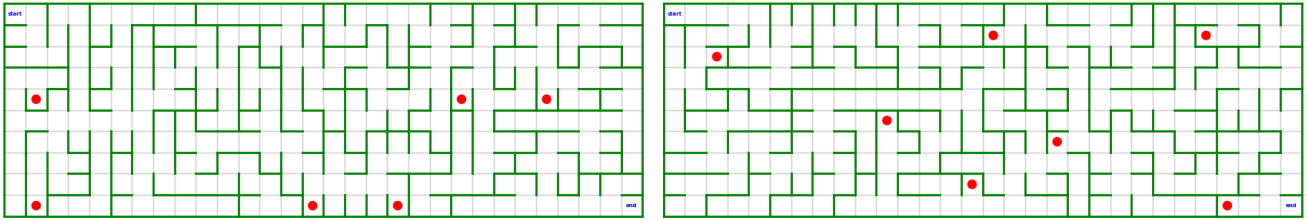
8. Beispiel: labyrinth7.txt, $n = 30, m = 10$ 

Abbildung 8: Visualisierung der Eingabe labyrinth7.txt

Diese Eingabe hat keine Loesung, weil es im zweiten Labyrinth kein Weg vom Start zum Ziel gibt, wenngleich es im ersten Labyrinth einen solchen Weg gibt.

9. Beispiel: labyrinth8.txt, $n = m = 150$

Die Approximation erreicht 716 Anweisungen, waehrend die optimale Loesung 472 Anweisungen erreicht.

Approximation:

```

DRRDDDDRRDDRDRRDDRDRRRRDRRURDRDLDDRDDDRDDDRRR
DDRDRDRDRDRDDDDRRDDDRDDDDLDDRRDLDRRRRRDDDDDRD
RDRRRDDLDDLDDDDLDRRDDRDRRRURDDRDRDDDRDLDDDDDDL
DDLDDLDDLDDRDRRRRRRDDDDRRDRDDRDRDDLDRRRRRDDDDDR
DRURDDRDRRRRRRRRRRDRDDDDRRDRRRDDDDLDDLDDDDDDLDR
DDLDDDDLDDLDDLDRRRDDDDDDDRDDRDDDDDRRRDRRRDDDDDR
DRDDRURRUUUURRRURURRRDRDDRURRDRRRDRRUURRRRUU
URDDRDRDRDRDRDRRRDRRRURRRDRUUUUURRURURURURDR
URRRURRRRRRRRRURRRRRRRRRDRDDDRDRRRRRDDRDRDDDR
RULUUURRRDRDDRDRRRDRRRDRDDRDRDDDRDDDLDDRDRDDDR
DRRRRRRDDRDRDRRRDRDDRDRRRRDDDDRRRRDDRDRRRDDR
DDDDDRDRDRDRDDDDDDLDRDRDRDRRRRRDRDRRRDDDDDDDRD
DDDRDRURDDRDRDDRDRDRDRDRDDRDDRDRRRDRDDDRRRR
URRRRRRRDRDDRDDRDRDDRDRRRUUUURDDRDRRRRRDRRRRR
DDDDDDRRRRDRRUURRRDRDDDDRRDDRDRRRDRURDDRDRDDR
RDDRDRDDDDLDDLDDDDLDRDRDDRDRRRDRDDDDLDRDDDDDDDR

```

Optimale Loesung:

```

DRRDDDDRRDDRDDDRDDRDRRRDRRRRDRRURDRDLDRDRDRDDDR
RRRDDDRRRRUURRRDRDDRURRRRRRRDDRDRRRRRDRDRRRDD
RRDDDRRRRDDLDRLDDDDDRDRDRDRDRDRDRDDDRRRDRDDDR
DRDDDRDRDDLDRDRDRDRDRDRURRDRRRRRRRDDDDDDDDDRD
DDRDDRURDDRDRDDRURDRRRDRUUUUULDDLUUULURURURDRD
RRRDRDDDDRDRRRDRDRRRDRDRRRURRRDRRRDRDDDDDRDLDDR
DRRDDLLDDRDRRRUUURRUURRRDRDRURRRRRURRRDRRRDDDRD
DDRDLULLLLDDLDDLDRDLDDLDDRDDDRDRDRRUURRRURDRD
DDRDRDRURDDRDDDDDRDRRRDRDRRRDDDDLULLDDLDRDR
RDDLLUDLLDLLLDRDRDDRDRURRRRRRRRRRRDRDDRDRRRRDL
LDRRDDDDDDDRURDDR

```

10. Beispiel: labyrinth9.txt, $n = m = 170$

Die Approximation erreicht 1377 Anweisungen, waehrend die optimale Loesung 1012 Anweisungen erreicht.

Approximation:

```

DRDRRRURURRDRRURDRRRDLDDLDDDDDDDRURRRUUURURDDR
DDRDRRRDDRURDDRDLDDLDDLDRDDRDRDDDDDDDRDDRDRD
DRURRRRRRRDRRRRRDDLDRURDDRDRDDLLDDDLDRRRRRRRDR
URDRDRDRURRRRRRRDDDRDRRRURURURUUURRRRRDRDLDRRR
RDRDRDRDDRURUUURURURRRDRDRDRDRDRDDRURUUURRRDR
RDLDDRDDRURURRRRRRRRRDRUUUUUUUUURDDRDRRUURR

```

```

URRURRUURRDRRDDRDRDDRDRRRRDDDDDDDDDRDRURDRDRDDR
RDRDLDDDLDLUULDDDDLDDLDRDDDRDRDRDDRDRDRDRDRDRRRD
RDDDRDRDLDDRDRRRDRRDDDDLDDDLDRRRDLDDDDLDDDDLDDL
ULLLLULDLDDLDDRDRRRRDDDDDDDRDRRURDRDDRURUURURRU
RDRRDDDDDDRRDDDDLDDLDDRDRDDDDDDDDDRDLDDDDLDDL
UUULDLDDRDRDDDDLDDRDLDRDDDDDDDDDRDRRDDRDLDDRURUUR
DRDRURDRDRRRDRUUURURDRRDDDDDDRRRURRDDDDDDLUUUULLL
LLDDRDRDDLDDLDDRDRRRURRDDDDDDDDLDDRDRDRURURRU
URURUURRDDRDDRURRRDDDDDRDLDDLDDRDRDLDDRDRRRDL
LLDRDLDDLULULLDDRDRDLDDDDDRDDRDRDDLDDLDDLULLDD
LLULLDDDLDDRDRDLDDLDRDRDRRRRRURRRRURRUULULURR
RRRRRRRUURRDDDDRRRDRDDDRDDRLDDLDDLDDDDDRDDLULL
DDDDDDDDDDLDDLDDRDRDRDLDRDDLDDLDDLDDLDDRDRRRDD
DRDRDDDRRRUURRUUUUURRURRRULURURRURRULULLURRRDR
DDRURURRDRDDRDDDDLULUULLDRDDLDDLDDDDDRDLDDLDDR
DLDDRDRRRDRRRRUUURRDRRRUULULURRRDRDRDRDDRURUUR
UUULUUULURRRURRRURDRDLDRURURUUUUULULURURRURDR
RDDRDRRDLDDLDDLDDRDRDRRRDLDRDRURRDRDDDDDDDDL
DLDDLDRDRRRRDRURRDDRUURURDRDDLDDRDRURDRDRDRD
DDRURDDDDDRDDRURUUULUURRDRURDRDRDRRRDDLDDL
RDRDLDDLDDLDDLDDLDDLDDRDRDRURRRRRURRDRDRDRDD
DDLDDRDRRRRDDRURUURDRULURRDRURRRRUUULUURURDRD
DRURRRRUUUUUURDRRRRRURURDDDRDDDDDRDRDRDRURRR
DRRDLDDLDDRDRDDRDLDRRDRUURRDRDRDRRUUURDDRDRDD

```

Optimale Loesung:

```

DRDRDRURRDDRURURRDDDDRRUURRDRDRDDRDDLDDLDD
RDDRDRRRURRDDDDDDLDDRDRURRURRUURURUURDRDDR
DDRURRDDRDRDRDLUDLLDRURDRDLDDRDRDLDDL
LUULLDLDDRDDRDLDDDDDRDDRDDLDDLDDLULULLDR
DLDDRDRDLDDLDRDRDRDRDRURRRURRUURLULURDRRR
RDRRRRUURDLDDLDRURUDRDDDRDDRDDLDDLDDLDDRDL
ULLDDDDDDDDLDDLDDRDRRRRRDRDRDLUDRURDDLDDLDDL
LLDRDRDRURRRDDDDDRDRDDDRDRRUURURURUUURRURDR
URLDDLDRURURRURDRULULLURDDRDRDRDDRURUURURDR
RDRDDRDRDRDRDRDLDDRRLUULLUDRRRDDLLLDDDDDRDL
DDRDLDDRDRRRDRRRRUUURRDRRRUULULURRDRRRDRDR
DRUUURRUUUULUUULURRRURDRURDDRDLDRRUUURRURRU
ULULURURUURURDRDRDDRDRDRDLDRRRRLDDLDDLDRDR
DRURDRDLDRDRURRDRDDRULDDLDDLULDDLDDLDRDR
RRLDRDRURDRDDRURURDRDLDDRDRURDRDRDDRURUDR
DDDDLDRDRDRURUUULUURRDRURDRUDRDDRDRDDLDDR
RDRDLDDLDDLDDLDDLULULLDDLDDLDDRDRDRURRRR
RURDLRDRDDDDDDLDRDRURRDRDDRURUURDRULURRDRUR
RRRUUUULUURURDRDDRURRRRRDDDDRUUUUUURDDLDDLDDR
RRDRDRURURDDRDDDDDRDRDRDLDDRURRRDRRRDLRLUUL
UUULDDLDDRDRDDLDDRDLDRDDDDDDDRDRURDDRDLDDR
UUURDRDRURDRDRURDRUUURURDRRDDDDDRRRURRDDDD

```

7.3. Fazit der Evaluierung

Anhand der Beispiele erkennt man, dass das Approximationsverfahren mittelmaessige Ergebnisse erzielt, aber dafuer eine extremst schnelle Laufzeit hat. Waehrenddessen hat das Verfahren fuer optimale Loesungswege eine sehr hohe Laufzeit, und ebenfalls einen sehr hohen Speicherverbrauch, sodass insbesondere eine Eingabe nicht geloest werden konnte. Dementspraechend waere es lohnenswert weitere Approximationsverfahren basierend auf dem optimalen Verfahren zu entwickeln.

8. Quellcode

Nun folgt der Quellcode der wichtigsten Teile der Loesung in C++. Dabei werden zunaechst gemeinsame Bestandteile der beiden Loesungsverfahren, wie die beiden structs *Coordinate* und *Maze* angegeben. Daraufhin folgen die spezifischen Implementierungen der Approximationsloesung und der optimalen Loesung.

8.1. Gemeinsame Bestandteile

Coordinate struct

Im Code beider Loesungsverfahren wird ein struct *Coordinate* verwendet, welches Positionen in einem Labyrinth mit ihrer Zeile und Spalte speichern sollen.

```

1 // Coordinate.h
2 // Struct storing two integer variables used for fields in the maze.
3 // Espacially enabling operations like addition, subtraction and comparison.
4 struct Coordinate {
5     int row;
6     int column;
7
8     Coordinate();
9     Coordinate(int row, int column);
10
11     Coordinate operator+(const Coordinate &other) const;
12     Coordinate operator-(const Coordinate &other) const;
13
14     bool operator<(const Coordinate& other) const;
15     bool operator==(const Coordinate& other) const;
16     bool operator!=(const Coordinate& other) const;
17
18     friend std::ostream& operator<<(std::ostream& os, const Coordinate& coord);
19 };
20
21 // Coordinate.cpp
22 // Implementation of Coordinate.h
23 Coordinate::Coordinate() {
24     Coordinate(0, 0);
25 }
26
27 Coordinate::Coordinate(int row, int column) : row(row), column(column) { }
28
29 // Function beforming elementwise addition, returning a new Coordinate object
30 Coordinate Coordinate::operator+(const Coordinate &other) const {
31     return {row + other.row, column + other.column};
32 }
33
34 // Function beforming elementwise subtraction, returning a new Coordinate object
35 Coordinate Coordinate::operator-(const Coordinate &other) const {
36     return {row - other.row, column - other.column};
37 }
38
39 // Function comparing Coordinates, such that first row will be compared, and for
40 // equality of row values, column value will be compared
41 bool Coordinate::operator<(const Coordinate& other) const {
42     if (row == other.row) {
43         return column < other.column;
44     }
45     return row < other.row;
46 }
47
48 // Simple == implementation, returning whether both Coordinates have the same row and column values
49 bool Coordinate::operator==(const Coordinate& other) const {
50     return row == other.row && column == other.column;
51 }
52
53 // Simple != implementation based on the == implementation
54 bool Coordinate::operator!=(const Coordinate& other) const {
55     return !(*this == other);
56 }
57
58 // Function writing the object to some outputstream
59 std::ostream& operator<<(std::ostream& os, const Coordinate& coord) {
60     return os << coord.row << " " << coord.column;
61 }

```

```
61 }
```

Implementierung der Darstellung eines Labyrinths

Ein weiteres struct *Maze* soll ein einzelnes Labyrinth darstellen und enthaelt insbesondere eine Funktion fuer die umwandlung des Labyrinths in dessen Graphen.

```
1 // Maze.h
2 // Struct storing a maze, containing 2-dimensional vector of holes, and
3 // 4 2-dimensional boolean vectors in an array, containing the wall
4 struct Maze {
5     public:
6         const int rows;
7         const int columns;
8
9     private:
10        // Each 2-dimensional vector has values 0 or 1, such that
11        // vector[i][j] is 1 if there is a wall in the certain direction
12        // at the field in row i and column j
13        std::array<std::vector<std::vector<bool>>, 4> walls;
14
15        // 2-dimensional vector having values 0 or 1, such that
16        // holes[i][j] is 1 if there is a hole at the field in row i and column j
17        std::vector<std::vector<bool>> holes;
18
19        // Start and end fields of the maze
20        const Coordinate start;
21        const Coordinate end;
22
23    public:
24        Maze(int rows, int columns, const Coordinate& start, const Coordinate& end);
25
26        // Function returning a vector of the given node's four neighbours
27        std::vector<Coordinate> get_neighbours(const Coordinate& node) const;
28
29        // Constructs a graph from the maze
30        std::vector<std::vector<Coordinate>> construct_graph() const;
31
32        // Returns the new coordinate when going from certain coordinate in a given direction
33        Coordinate step(const Coordinate& coord, int direction) const;
34
35        // Changes the array walls such that it indicates that there is a
36        // wall in the certain direction at the given field
37        void set_wall(int direction, int row, int column);
38
39        // Changes the holes vector, such that it indicates that there is
40        // a hole at the given field
41        void set_hole(int row, int column);
42
43        // Return weather there is a wall in the given direction on the given field
44        bool has_wall(int direction, int row, int column) const;
45
46        // Returns weather there is a hole at the given field
47        bool has_hole(int row, int column) const;
48
49        // Converts an coordinate of the maze to its index
50        int to_index(const Coordinate& coord) const;
51 };
52
53 // Maze.cpp
54 // Implementation of Maze.h
55 // Define the indices for top, right, bottom and left as well as the offsets in an xy-grid
56 const int UP = 0, RIGHT = 1, DOWN = 2, LEFT = 3;
57 const Coordinate offset[4] = { Coordinate(-1, 0), Coordinate(0, 1), Coordinate(1, 0), Coordinate(0, -1) };
58
59 // Constructor initialising all fields of the struct.
60 // Espacially setting up empty 2-dimensional vectors for holes and walls
61 Maze::Maze(int rows, int columns, const Coordinate& start, const Coordinate& end)
62 : rows(rows), columns(columns), start(start), end(end), holes(rows, std::vector<bool>(columns, false))
63 for (int i = 0; i < 4; i++) {
64     walls[i] = std::vector<std::vector<bool>>(rows, std::vector<bool>(columns, false));
65 }
```

```

67 }

69 // Function returning a vector of the four fields (Coordinates) one will reach
// when doing one step to top, right, bottom, left, assuming that the given field
71 // is not a hole.
std::vector<Coordinate> Maze::get_neighbours(const Coordinate& coordinate) const {
73     if (Maze::has_hole(coordinate.row, coordinate.column)) return {};

75     std::vector<Coordinate> edges(4);

77     // For each of the four directions find the next field
    for (int direction = 0; direction < 4; direction++) {
79         edges[direction] = Maze::step(coordinate, direction);
    }

81     return edges;
83 }

85 // Construct the maze corresponding to a maze. That is, the graph having the rows*columns
// Coordinates from the maze as nodes and edges corresponding to moving in one of the four directions
87 // from each Coordinate
std::vector<std::vector<Coordinate>> Maze::construct_graph() const {
89     // Create the adjacency list for the graph described above
    int size = Maze::rows * Maze::columns;
91     std::vector<std::vector<Coordinate>> adjacency(size);

93     for (int row = 0; row < Maze::rows; row++) {
        for (int column = 0; column < Maze::columns; column++) {
95             const Coordinate coordinate(row, column);
            adjacency[Maze::to_index(coordinate)] = Maze::get_neighbours(coordinate);
97         }
    }

99     return adjacency;
101 }

103 // Returns the new coordinate when going from certain coordinate in a given direction
Coordinate Maze::step(const Coordinate& coord, int direction) const {
105     // If there is a wall in the current direction, the field will not change
    if (coord == Maze::end || Maze::has_wall(direction, coord.row, coord.column)) {
107         return coord;
    }

109     // If there is no wall in the current direction, the next field will either
// be the one you will reach by going on step in the current direction,
// or the start field, if there is a hole on the later field.
111     Coordinate offset_coord = coord + offset[direction];
    Coordinate next_coord
113     = Maze::has_hole(offset_coord.row, offset_coord.column) ? Maze::start : offset_coord;
    return next_coord;
115 }

117 }

119 // Function converting a coordiante in a maze to an index,
// mapping each coordinate to a unique index in [0, rows*columns)
121 int Maze::to_index(const Coordinate& coord) const {
    return coord.row * Maze::columns + coord.column;
123 }

125 // Changes the array walls such that it indicates that there is a
// wall in the certain direction at the given field
127 void Maze::set_wall(int direction, int row, int column) {
    walls[direction][row][column] = true;
129 }

131 // Changes the holes vector, such that it indicates that there is
// a hole at the given field
133 void Maze::set_hole(int row, int column) {
    holes[row][column] = true;
135 }

137 // Return whether there is a wall in the given direction on the given field
bool Maze::has_wall(int direction, int row, int column) const {
139     return walls[direction][row][column];
}

141 // Returns whether there is a hole at the given field

```

```

143 bool Maze::has_hole(int row, int column) const {
    return holes[row][column];
145 }

```

8.2. Implementierung des Approximationsverfahrens

Nun gehen wir auf die wichtigsten Teile der Implementierung des Approximationsverfahrens ein.

Hauptteil

Der relevante Teil der Hauptfunktion des Approximationsverfahrens sieht wie folgt aus:

```

1 // Create the graph of both mazes
  const std::vector<std::vector<Coordinate>> adjacency1 = maze1.construct_graph();
3 const std::vector<std::vector<Coordinate>> adjacency2 = maze2.construct_graph();

5 // Check weather the end field is reachable from the start field in both mazes
  const std::vector<bool> visible1 = get_visible_coordinates(start, adjacency1);
7 bool is_possible_1 = visible1[to_index(end)];

9 const std::vector<bool> visible2 = get_visible_coordinates(start, adjacency2);
  bool is_possible_2 = visible2[to_index(end)];

11 // If the start and end fields are not connected in both mazes, stop the program
13 if (!(is_possible_1 && is_possible_2)) {
    // [...]
    return;
15 }

17 // Otherwise run the approximation algorithm in both possible orders
19 std::vector<int> directions1 = combine_paths(adjacency1, adjacency2);
  std::vector<int> directions2 = combine_paths(adjacency2, adjacency1);

21 // Make sure directions1 has fewer amount if directions, because directions1 will be printed
23 if (directions2.size() < directions1.size()) {
    std::swap(directions1, directions2);
25 }

```

Vorher wurden die zwei Labyrinth in den Objekten *maze1* und *maze2* eingelesen. In diesem Codeabschnitte ist die entgültige Bewegungsfolge am Ende im vector *directions1* gespeichert. Daraufhin werden die Ergebnisse ausgegeben. Die hier benutzen Funktionen sind wie folgt implementiert:

```

1 // Const for infinity in shortest path
  const int INFITY = 1e9;

3 // Define the indices for up, right, bottom and left as well as the offsets in an xy-grid
5 const int UP = 0, RIGHT = 1, DOWN = 2, LEFT = 3;
  const Coordinate offset[4] = { Coordinate(-1, 0), Coordinate(0, 1), Coordinate(1, 0), Coordinate(0, -1) };

7 // Globally store the amount of rows and columns in both mazes
9 int rows, columns;

11 // Globally store start and end fields
  Coordinate start, end;

13 // Function converting a coordiante in a maze to an index,
  // mapping each coordinate to a unique index in [0, rows*columns)
15 int to_index(const Coordinate& coord) {
    return coord.row * columns + coord.column;
17 }

19 // Given the graph of a maze as its adjacency list, as well as a starting coordiante,
21 // compute a vector booleans indicating weather or not you can reach some coordinate from the given
  // starting coordiante in the given maze
23 std::vector<bool> get_visible_coordinates(const Coordinate& starting_field,
    const std::vector<std::vector<Coordinate>>& adjacency) {
25 // Perform a default breath-first search
  std::vector<bool> visit(rows * columns, 0);
  std::queue<Coordinate> queue;
  queue.push(starting_field);
27 }

```

```

29
30 // Repeat until the queue is empty
31 while (queue.size() > 0) {
32     // Get the current coordinate from the queue, remove it from the queue and get its index
33     Coordinate node = queue.front();
34     queue.pop();
35     int index = to_index(node);
36
37     // If coordinate (node) was already visited, dont visit it again, otherwise mark it as visited
38     if (visit[index]) continue;
39     visit[index] = true;
40
41     // Add all neighbours of the current coordinate (node) to the queue
42     for (Coordinate next : adjacency[index]) {
43         queue.push(next);
44     }
45 }
46
47 return visit;
48 }
49
50 // Compute the single-source shortest path for the given unweighted graph using a simple bfs,
51 // writing the results, e.i., distances and predecessor for each node to given vectors.
52 // Assuming dist vector has initially all values set to infinity
53 void shortest_path(std::vector<int>& dist, std::vector<Coordinate>& last,
54     const Coordinate& starting_field,
55     const std::vector<std::vector<Coordinate>>& adjacency) {
56     // Perform a default breath-first search
57     // The queue contains the the current cost, current node and predecessor node
58     std::queue<std::tuple<int, Coordinate, Coordinate>> queue;
59     queue.push({0, starting_field, starting_field});
60
61     // Repeat until the queue is empty
62     while (queue.size() > 0) {
63         // Get the current node from the queue, remove it from the queue and compute its index
64         int distance;
65         Coordinate node, parent;
66         std::tie(distance, node, parent) = queue.front();
67         queue.pop();
68         int index = to_index(node);
69
70         // If coordinate (node) was already visited (by checking if dist value is already set),
71         // dont visit it again, otherwise set the values of dist and last
72         if (dist[index] != INFTY) continue;
73         dist[index] = distance;
74         last[index] = parent;
75
76         // Add all neighbours of the current coordinate (node) to the queue
77         for (Coordinate next : adjacency[index]) {
78             queue.push({distance+1, next, node});
79         }
80     }
81 }
82
83 // Given the last vector computed by a shortest paths algorithm, compute the path from start to end.
84 // Last contains for each coordinate (node of the graph) its predecessor.
85 std::vector<Coordinate> construct_path(const Coordinate& starting_field, const Coordinate& ending_field,
86     const std::vector<Coordinate>& last) {
87     // Return a path of length one if start = end
88     if (starting_field == ending_field) {
89         return {starting_field};
90     }
91
92     // Iteratively constructing the path backwards, starting at end
93     std::vector<Coordinate> path;
94     Coordinate node = ending_field;
95
96     do {
97         path.push_back(node);
98         int index = to_index(node);
99         node = last[index];
100     } while (node != starting_field);
101
102     // Finally add the start node
103     path.push_back(starting_field);

```

```

105     // Reverse the path to achieve the correct order (from start to end)
106     std::reverse(path.begin(), path.end());
107     return path;
108 }
109
110 // Given some path in the maze, compute the sequence of directions creating it
111 std::vector<int> construct_directions(const std::vector<Coordinate>& path) {
112     int n = path.size();
113     if (n <= 1) return {};
114
115     std::vector<int> directions(n-1);
116
117     for (int i = 0; i < n-1; i++) {
118         Coordinate node1 = path[i];
119         Coordinate node2 = path[i+1];
120
121         // Find out which directions to apply in order to get from node1 to node2 in the maze
122         for (int direction = 0; direction < 4; direction++) {
123             if (node1 + offset[direction] == node2) {
124                 directions[i] = direction;
125             }
126         }
127     }
128
129     return directions;
130 }
131
132 // Apply a sequence of directions to some starting field in a given maze
133 Coordinate apply_directions(const Coordinate& starting_field,
134                             const std::vector<std::vector<Coordinate>>& adjacency,
135                             const std::vector<int>& directions) {
136     Coordinate node = starting_field;
137
138     for (int d : directions) {
139         // Make sure to stop moving if the end field is reached
140         if (node == end) {
141             break;
142         }
143
144         // Move to the next field
145         int index = to_index(node);
146         node = adjacency[index][d];
147     }
148
149     return node;
150 }
151
152 // Given the graphs of the two mazes, combine the shortest paths from start to end in
153 // both mazes to one path working for both mazes.
154 std::vector<int> combine_paths(const std::vector<std::vector<Coordinate>>& adjacency1,
155                               const std::vector<std::vector<Coordinate>>& adjacency2) {
156     int size = rows * columns;
157
158     // Compute the shortest path from start to end in the first maze
159     std::vector<int> dist1(size, INFTY);
160     std::vector<Coordinate> last1(size);
161     shortest_path(dist1, last1, start, adjacency1);
162
163     // Create the path and the resulting list of directions from the shortest path in the first maze
164     std::vector<Coordinate> path1 = construct_path(start, end, last1);
165     std::vector<int> directions1 = construct_directions(path1);
166
167     // Apply the list of directions of the shortest maze in first maze two the
168     // second maze
169     Coordinate node_second = apply_directions(start, adjacency2, directions1);
170
171     // Compute the shortest path from the just computed node_second to end in second maze
172     std::vector<int> dist2(size, INFTY);
173     std::vector<Coordinate> last2(size);
174     shortest_path(dist2, last2, node_second, adjacency2);
175
176     // Create the path and the resulting list of directions from the shortest path in the second maze
177     std::vector<Coordinate> path2 = construct_path(node_second, end, last2);
178     std::vector<int> directions2 = construct_directions(path2);
179
180     // Compute the final sequence of directions working for both mazes by combining the two computed
181     // list of directions

```

```

181     std::vector<int> directions = directions1;
182     for (int direction : directions2) {
183         directions.push_back(direction);
184     }
185     return directions;
187 }

```

Insbesondere entspricht die Funktion *combine_paths* dem Hauptteil des Loesungsalgorithmus, in dem zwei kuerzesten Wege verbunden werden.

8.3. Implementierung des Verfahrens fuer optimale Loesungswege

Nun gehen wir auf die wichtigsten Teile der Implementierung des Approximationsverfahrens ein.

Node struct

Um fuer das Verfahren fuer optimale Loesungswege den Graphen der Zustaende $C \times C$ darzustellen, wird das struct *Node* benutzt:

```

1 // Node.h
2 // Struct for storing two coordiantes in one place, providing
3 // the usual comparision operations
4 struct Node {
5     Coordinate first;
6     Coordinate second;
7
8     Node();
9     Node(const Coordinate& first, const Coordinate& second);
10
11     bool operator<(const Node& other) const;
12     bool operator==(const Node& other) const;
13     bool operator!=(const Node& other) const;
14
15     friend std::ostream& operator<<(std::ostream& os, const Node& node);
16 };
17
18 // Node.cpp
19 // Implemenation of Node.h
20 Node::Node() {
21     Coordinate zero(0, 0);
22     Node(zero, zero);
23 }
24
25 Node::Node(const Coordinate& first, const Coordinate& second) : first(first), second(second) { }
26
27 // Implementation of < comparision with another Node based on comparison of Nodes
28 bool Node::operator<(const Node& other) const {
29     if (first == other.first)
30         return second < other.second;
31     return first < other.first;
32 }
33
34 // Basic implementation of == based on comparison of Nodes
35 bool Node::operator==(const Node& other) const {
36     return first == other.first && second == other.second;
37 }
38
39 // Basic implementation of != using the implementation of ==
40 bool Node::operator!=(const Node& other) const {
41     return !(*this == other);
42 }
43
44 std::ostream& operator<<(std::ostream& os, const Node& node) {
45     return os << node.first << "□" << node.second;
46 }

```

Hauptteil

Der relevante Teil der Hauptfunktion des Approximationsverfahrens sieht wie folgt aus:

```

// Find the shortest path
2 int size = rows*columns*rows*columns;
  std::vector<int> dist(size, INFTY);
  std::vector<Node> last(size);
  shortest_path(Node(start, start), maze1, maze2, dist, last);
6
// Construct the actual path from last vector
8 std::vector<Node> path = construct_path(Node(start, start), Node(end, end), last);

10 // Construct the directions creating the path
  std::vector<int> directions = construct_directions(path);

```

Vorher wurden die zwei Labyrinth in den Objekten *maze1* und *maze2* eingelesen. In diesem Codeabschnitte ist die entgeltige Bewegungsfolge am Ende im vector *directions* gespeichert. Daraufhin werden die Ergebnisse ausgegeben. Die hier benutzen Funktionen sind wie folgt implementiert:

```

1 // Const for infinity in shortest path
  const int INFTY = 1e9;
3
// Define the indices for up, right, bottom and left as well as the offsets in an row, column based Coordinate
5 const int UP = 0, RIGHT = 1, DOWN = 2, LEFT = 3;
  const Coordinate offset[4] = { Coordinate(-1, 0), Coordinate(0, 1), Coordinate(1, 0), Coordinate(0, -1) };
7
// Globally store the amount of rows and columns in both mazes
9 int rows, columns;

11 // Globally store start and end fields
  Coordinate start, end;
13

15 // Function converting a Coordinate in the maze to some index,
  // such that there will be a bijective mapping
17 int to_index(Coordinate& first, Coordinate& second) {
  int i1 = first.row * columns + first.column;
  int i2 = second.row * columns + second.column;
  return i1 * rows*columns + i2;
21 }

23 // Given a coordinate of each of the two mazes, computed the four possible next
  // states depending on the choosen direction.
25 std::vector<Node> get_edges(const Coordinate& first, const Coordinate& second,
  const Maze& maze1, const Maze& maze2) {
27   // If one of the coordinates is a field with a hole, we can ignore (because we can never land here)
  if (maze1.has_hole(first.row, first.column)) return {};
  if (maze2.has_hole(second.row, second.column)) return {};
29
  // Get the four neighbours of the given coordinates per maze
  std::vector<Coordinate> neighbours1 = maze1.get_neighbours(first);
  std::vector<Coordinate> neighbours2 = maze2.get_neighbours(second);
33
  // Combine the two vectors of neighbours
  std::vector<Node> reach(4);
37
  for (int i = 0; i < 4; i++) {
39     Node node = Node(neighbours1[i], neighbours2[i]);
    reach[i] = node;
41  }

43  return reach;
}

45
// Compute the single-source shortest path for a unweighted graph using a simple bfs,
// writing the results, e.i., distances and predecessor for each node to given vectors.
// Assuming dist vector has initally all values set to infinity
47 void shortest_path(const Node& starting_node, const Maze& maze1, const Maze& maze2,
  std::vector<int>& dist, std::vector<Node>& last) {
51   // Perform a default breath-first search
  // The queue contains the the current cost, current node and predecessor node
53   std::queue<std::tuple<int, Node, Node>> queue;
  queue.push({0, starting_node, starting_node});
55
  // Repeat until the queue is empty
57   while (queue.size() > 0) {
    // Get the current node from the queue, remove it from the queue and compute its index
59     int distance;

```

```

Node node, parent;
61 std::tie(distance, node, parent) = queue.front();
   queue.pop();
63 int index = to_index(node.first, node.second);

65 // If node was already visited (by checking if dist value is already set),
   // dont visit it again, otherwise set the values of dist and last
67 if (dist[index] != INFTY) continue;
   dist[index] = distance;
69 last[index] = parent;

71 // Add all neighbours of the current node to the queue
   for (Node next : get_edges(node.first, node.second, maze1, maze2)) {
73     queue.push({distance+1, next, node});
   }
75 }
77 }

// Given the last vector computed by a shortest paths algorithm, compute the path from start to end.
79 // Last contains for each coordinate (node of the graph) its predecessor.
std::vector<Node> construct_path(const Node& start_node, const Node& end_node, std::vector<Node>& last) {
81     // Return a path of length one if start = end
   if (start_node == end_node) {
83         return {start_node};
   }

85     // Iteratively constructing the path backwards, starting at end
   std::vector<Node> path;
   Node node = end_node;

87     do {
91         path.push_back(node);
         int index = to_index(node.first, node.second);
93         node = last[index];
   } while (node != start_node);

95     // Finally add the start node
   path.push_back(start_node);

97     // Reverse the path to achieve the correct order (from start to end)
   reverse(path.begin(), path.end());
101    return path;
}

103 // Given some path in both mazes, compute the sequence of directions creating it
105 std::vector<int> construct_directions(const std::vector<Node>& path, const Maze& maze1, const Maze& maze2) {
   int n = path.size();
107   std::vector<int> directions(n-1);

109   for (int i = 0; i < n-1; i++) {
       Node node1 = path[i];
111       Node node2 = path[i+1];

113       // Find out which directions to apply in order to get from node1 to node2 in the mazes
       for (int direction = 0; direction < 4; direction++) {
115           if ((maze1.step(node1.first, direction) == node2.first) &&
               (maze2.step(node1.second, direction) == node2.second)) {
117               directions[i] = direction;
           }
       }
119   }
121   return directions;
123 }

```